M02 — Formularios, validación y Razor Pages en ASP.NET Core MVC

Teoría del módulo 2 — Formularios y persistencia de productos

CURSO: Desarrollo Web con ASP.NET Core MVC, Razor y Blazor en .NET 8

TIPO: TEORÍA

AUTOR: JAIME GALLO

Este manual continúa directamente el catálogo de productos construido en M01. El alumno ya sabe relacionar una solicitud HTTP con una ruta MVC, seleccionar una acción del controlador, entregar un modelo tipado a una vista Razor y reutilizar un layout. El nuevo problema no es mostrar datos existentes, sino recibir información que escribe otra persona, entender cómo se transforma en objetos C#, decidir si puede aceptarse y, finalmente, conservarla.

La progresión acordada para este módulo abarca cinco puntos: **2.1** formularios HTML, GET/POST, model binding y antiforgery; **2.2** Tag Helpers; **2.3** DataAnnotations, ModelState y validación; **2.4** CRUD con EF Core y SQL Server Express LocalDB; y **2.5** una comparación breve con Razor Pages dentro del mismo host MVC. El conjunto de capítulos se presenta de forma progresiva en este mismo manual, con el código fuente de cada práctica situado en la carpeta correspondiente.

Al finalizar el día, el producto será un catálogo que permita crear, consultar, modificar y eliminar datos persistidos. La aplicación Razor Pages será solamente una demostración alternativa de recepción de información, no otro proyecto empresarial. Seguimos usando **.NET 8** y los conocimientos previos de C#, Web API y Entity Framework Core; no volveremos a impartir esas tecnologías desde el principio.

Índice del módulo

- 2.1. Formularios HTML, GET, POST, model binding y antiforgery.
- 2.2. Tag Helpers de formularios, campos y enlaces.
- 2.3. DataAnnotations, ModelState y validación.
- 2.4. CRUD persistente con EF Core y LocalDB.
- 2.5. Comparación de MVC y Razor Pages.

2.1. Formularios HTML, GET y POST, model binding y antiforgery

El problema que resuelve un formulario

Hasta ahora, el usuario de nuestro catálogo podía navegar entre la portada, la lista de productos y una ficha individual. Las tres posibilidades pertenecían, principalmente, al mundo de las **consultas**. Una URL como `/Productos/Detalle/2` permite identificar qué información queremos recuperar; el servidor determina qué objeto `Producto` corresponde al identificador y genera una página HTML.

Para introducir información nueva necesitamos que el navegador envíe valores al servidor. Imaginemos que un empleado de una papelería necesita comunicar el nombre y el precio de un producto que todavía no aparece en el catálogo. No tiene sentido modificar el código fuente y volver a compilar cada vez. La página debe proporcionar controles de edición y un mecanismo para remitir esos valores a ASP.NET Core MVC. Ese mecanismo es el **formulario HTML**.

Un formulario no es, por sí solo, una operación de base de datos. Es un contrato de entrada entre el navegador y un destino HTTP. Nuestra primera versión únicamente recibirá un nombre y un precio y los volverá a mostrar en una página de confirmación. Esa decisión es intencionada: si incorporáramos SQL Server en el primer ejemplo, sería difícil distinguir los errores de envío y enlace de datos de los errores de almacenamiento. La persistencia aparecerá en 2.4, una vez comprendidos y comprobados los pasos anteriores.

Un mismo recurso web puede responder de forma distinta según el método

En M01 aprendimos que el enrutamiento determina qué código puede atender una URL. Esa explicación necesita ahora una precisión: **no basta con la URL; también importa el método HTTP**.

El navegador puede solicitar la página del formulario mediante:

```
GET /Productos/Crear HTTP/1.1
Host: localhost:5091
Accept: text/html
```

Una solicitud GET significa aquí que queremos obtener una representación del formulario. El servidor utiliza la acción marcada con `[HttpGet]`, crea un objeto inicialmente vacío y selecciona la vista `Crear.cshtml`. El navegador recibe HTML: etiquetas `label`, controles `input`, botón de envío y un campo oculto para la protección antifalsificación.

Cuando el usuario pulsa el botón, el navegador puede enviar otra petición a **la misma ruta**, esta vez con método POST:

```
POST /Productos/Crear HTTP/1.1
Host: localhost:5091
Content-Type: application/x-www-form-urlencoded

Nombre=Rotulador&Precio=7
```

Este segundo mensaje es deliberadamente simplificado: una solicitud aceptada por la aplicación incluirá también un token antiforgery y normalmente una cookie asociada, además de otras cabeceras. El ejemplo ilustra una diferencia fundamental: los valores se transportan como **datos de una petición**, no como el nombre de un archivo `.cshtml` ni como parámetros de código que se ejecuta en el navegador.

El método POST no significa automáticamente «guardar en la base de datos». Indica que el navegador está enviando información al endpoint para que el servidor la procese. Qué hace después la acción es una decisión del programa. En nuestro punto 2.1 la acción solo devuelve una vista de confirmación. En 2.4 ese POST validado también creará una entidad persistente.

Para el alumnado con experiencia en Web API, esta separación resultará familiar: distintas operaciones HTTP pueden actuar sobre recursos relacionados. Aquí la novedad es que la petición procede de una **interfaz HTML Razor**, y que el resultado puede ser otra página HTML en lugar de una respuesta JSON.

Leer el formulario real antes de construir su POST

El proyecto del curso incluye una vista `Views/Productos/Crear.cshtml` que podemos estudiar en términos de HTML ordinario:

```
@model CursoWeb.Mvc.Models.Producto
@{
    ViewData["Title"] = "Crear producto";
}
```

```
<h1>Crear producto — P2.1</h1>
<form method="post" action="/Productos/Crear">
    @Html.AntiForgeryToken()
    <div>
        <label for="Nombre">Nombre</label>
        <input id="Nombre" name="Nombre" value="@Model.Nombre" />
    </div>
    <div>
        <label for="Precio">Precio</label>
        <input id="Precio" name="Precio" type="number" step="0.01"

value="@Model.Precio.ToString(System.Globalization.CultureInfo.InvariantCulture)" />
    </div>
    <button type="submit">Enviar producto</button>
</form>
```

Es importante distinguir tres responsabilidades. El elemento `form` declara **cómo y adónde enviar** los datos mediante `method="post"` y `action="/Productos/Crear"`. Las etiquetas `input` describen **qué campos** puede modificar el usuario y qué nombres se enviarán. Las expresiones Razor, por ejemplo `@Model.Nombre`, se evalúan **en el servidor** cuando se genera el HTML inicial. El navegador recibe una página ya procesada, no una instrucción para ejecutar C#.

La etiqueta `name="Nombre"` es especialmente importante: el model binder utilizará ese nombre para asociar el valor enviado con la propiedad `Nombre` del objeto de entrada. `id="Nombre"`, en cambio, identifica el control dentro del documento HTML y permite vincularlo con la etiqueta `label for="Nombre"`. En nuestro primer formulario ambos atributos tienen la misma cadena, pero cumplen funciones distintas.

El atributo `type="number"` ofrece un control apropiado para cantidades numéricas. `step="0.01"` facilita la introducción de precios con decimales. No debemos interpretarlo como una validación suficiente para un servidor de producción: las peticiones HTTP pueden construirse sin utilizar el formulario ni su interfaz. La validación que define qué valores aceptaremos se estudiará en 2.3.

En este primer ejemplo expresamos el valor inicial del precio usando `CultureInfo.InvariantCulture`. Esto evita introducir un número con coma en el atributo HTML `value` que espera un formato numérico compatible con el control. **No resuelve por sí solo la globalización del enlace de modelos en todos los países**: una aplicación destinada a distintos formatos regionales debe configurar y comprobar su cultura de solicitud. Para el laboratorio reproducible utilizaremos precios enteros al comprobar el transporte inicial y trataremos los límites de validación más adelante.

El campo oculto que también envía el navegador

La vista contiene `@Html.AntiForgeryToken()`, que genera un elemento de entrada oculto con un valor de comprobación:

```
<input name="__RequestVerificationToken" type="hidden"
       value="un-valor-generado-por-el-servidor" />
```

El valor mostrado es una representación didáctica; el token real no es una contraseña escrita por nosotros ni debe copiarse de este ejemplo. ASP.NET Core lo genera al producir la página. El navegador también conserva la cookie pertinente. Cuando el formulario se envía, el servidor puede verificar que la solicitud incorpora la información antiforgery que corresponde.

¿Por qué se escribe el token de forma explícita en este formulario? Porque hemos utilizado un atributo HTML `action` con una URL literal. Es conveniente no depender aquí de condiciones implícitas de generación del token: `@Html.AntiForgeryToken()` hace visible la decisión de proteger el POST. En el apartado 2.2 sustituiremos progresivamente la construcción manual por **Form Tag Helpers**, que normalmente generan ese token de manera automática cuando se utilizan como corresponde. Consulta también la documentación oficial de Microsoft sobre antiforgery.

El controlador: dos acciones llamadas Crear

Una característica de MVC es que un controlador puede contener dos métodos C# con el mismo nombre de acción, diferenciados mediante atributos de método HTTP. En nuestro proyecto, el código GET está disponible desde el principio:

```
[HttpGet]
public IActionResult Crear() => View(new Producto());
```

Al navegar a `/Productos/Crear`, MVC selecciona este método para GET. `View(new Producto())` prepara la representación de la vista `Crear.cshtml` con un objeto cuyo `Nombre` está vacío y cuyo `Precio` es cero. Todavía no se ha recibido ningún producto nuevo. Crear un objeto C# para presentar el formulario no equivale a guardar un registro.

El proyecto incluye también **comentada** la acción POST. El alumno la activará después de comprobar el GET y el comportamiento anterior a la existencia del POST:

```
[HttpPost]
[ValidateAntiForgeryToken]
```

```
public IActionResult Crear(Producto producto)
{
    return View("Confirmacion", producto);
}
```

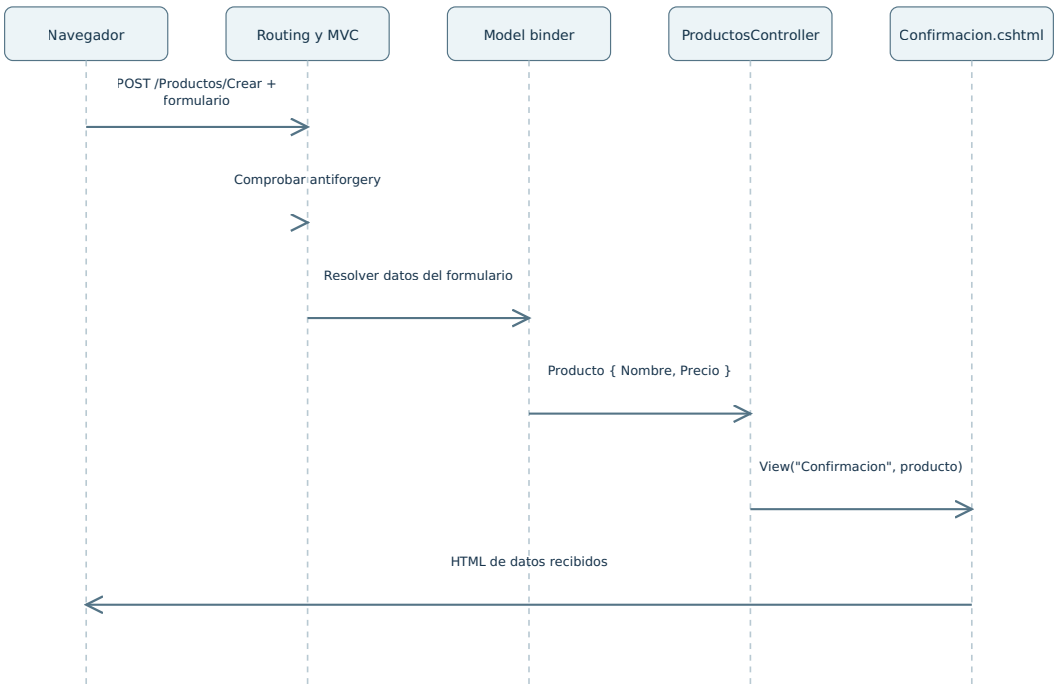
La presencia de `[HttpPost]` restringe esa acción al método POST. `[ValidateAntiForgeryToken]` solicita al framework comprobar el token antes de permitir que el método procese la solicitud. Finalmente, el parámetro `Producto producto` recibe un objeto rellenado por el model binder con los valores del formulario.

Este diseño evita una confusión muy común: la cadena `Nombre=Rotulador` no es código C# ni una instrucción para cambiar la propiedad directamente desde el navegador. Es texto transportado por HTTP. ASP.NET Core interpreta los campos de la solicitud y construye el objeto de entrada. La acción trabaja con ese objeto **después** de haber tenido lugar el enlace de modelos.

Cómo actúa el enlace de modelos

Podemos narrar el recorrido sin convertirlo en una lista de mecanismos internos que el alumno deba memorizar:

1. El usuario introduce `Nombre=Rotulador` y `Precio=7` en el navegador.
2. Al enviar el formulario, el cliente crea una petición POST con esos campos y la información antiforgery.
3. MVC identifica la acción `Crear(Producto producto)` correspondiente al método y a la ruta.
4. El model binder toma los valores que corresponden a las propiedades del tipo `Producto`. Intenta convertir `Precio` al tipo decimal de la clase.
5. La acción ejecuta `View("Confirmacion", producto)`; Razor usa ese objeto para generar el HTML final.



El diagrama muestra responsabilidades lógicas dentro del servidor; el binder no es un servicio remoto. Es igualmente importante observar **qué no ocurre**: no hay una instrucción `SaveChanges()`, ningún objeto `DbContext` ni escritura SQL. La vista de confirmación no puede demostrar que el producto haya quedado persistido. En realidad, basta con volver al catálogo para comprobar que conserva los tres productos originales.

La clase `Producto` procede del módulo anterior. No necesitamos estudiar de nuevo qué significa una propiedad automática en C#. Su función aquí es ser el **destino tipado** de los valores recibidos. Más adelante distinguiremos el modelo del dominio de un modelo específico para la entrada de formulario, de modo que no tengamos que aceptar todas las propiedades de la entidad desde una solicitud externa.

Antiforgery: por qué el POST requiere una protección adicional

Cuando un navegador conserva cookies de una aplicación, otras páginas podrían intentar inducirlo a enviar solicitudes no deseadas utilizando ese contexto. Los ataques de falsificación de solicitud entre sitios, conocidos como CSRF, aprovechan esa posibilidad. ASP.NET Core proporciona un mecanismo antiforgery basado en tokens para dificultar el envío de formularios falsificados desde contextos no autorizados.

Nuestra acción incluye `[ValidateAntiForgeryToken]` para que el procesamiento del POST dependa de la comprobación del token. En una navegación normal, la vista genera

el campo oculto; el navegador conserva la cookie y envía ambos valores cuando corresponde. Si realizamos el POST sin el token exigido, el framework puede devolver **400 Bad Request** antes de que se ejecute el cuerpo de la acción.

No significa que el token valide los datos del negocio. La protección antiforgery responde a «¿está autorizada esta forma de enviar la petición desde la sesión o contexto correspondiente?», pero no a «¿es correcto el precio del producto?» ni «¿hay otro producto con ese nombre?». La segunda familia de preguntas pertenece a la validación del modelo y al dominio.

Asimismo, antiforgery no sustituye autenticación, autorización, HTTPS ni un diseño seguro de permisos. En este curso trabajamos únicamente el escenario local necesario para comprender una solicitud HTML; no construiremos un sistema de Identity desde cero.

Lo que veremos cuando falte el POST

Antes de activar la acción `[HttpPost]`, el GET puede devolver correctamente el formulario y un envío POST a la misma dirección no encontrará una acción compatible. Es razonable obtener una respuesta **405 Method Not Allowed**. Esto no significa que el HTML esté mal: revela que el servidor reconoce un camino para consultar la página, pero todavía no tiene el endpoint esperado para procesar ese método.

Después de activar el POST, una solicitud de formulario correctamente construida puede terminar en 200 y representar `Confirmacion.cshtml`. Una solicitud sin token válido puede terminar en 400. Las dos respuestas no son contradictorias: MVC distingue ruta, método, autorización del envío y, cuando esos controles se superan, el procesamiento del cuerpo.

No confundiremos un 404 (dirección sin endpoint correspondiente), un 405 (método no admitido para ese destino) y un 400 por antiforgery. En las prácticas se presentan como observaciones de la respuesta HTTP que el alumno podrá relacionar con una causa concreta.

Qué representa la página de confirmación

La vista `Views/Productos/Confirmacion.cshtml` declara el mismo tipo de modelo que la acción POST:

```
@model CursoWeb.Mvc.Models.Producto

<h1>Datos recibidos del formulario</h1>
<p id="producto-recibido">Nombre: <strong>@Model.Nombre</strong></p>
<p id="precio-recibido">Precio:
    <strong>@Model.Precio.ToString("0.00")</strong> €</p>
```

Este archivo no contiene la etiqueta `<form>` principal. Su responsabilidad consiste en mostrar el resultado de una operación previa. Como aprendimos en M01, **Razor genera el HTML en el servidor**. La expresión `@Model.Nombre` representa el dato recibido, y el motor Razor codifica como corresponde las cadenas que se insertan en HTML. Por eso debemos utilizar expresiones Razor ordinarias en lugar de concatenar HTML de procedencia desconocida y marcarlo como no codificado.

La confirmación es el lugar adecuado para un primer ejercicio de transferencia: el alumno escribirá una condición `@if` que presente una etiqueta adicional solamente cuando `Model.Precio >= 4m`. No deberá copiar la solución del catálogo de M01; tendrá que decidir dónde colocar la condición y cómo comprobarla con dos solicitudes de precios distintos.

El cambio será exclusivamente **visual**. Si un producto llega con `Precio=7`, la etiqueta aparecerá en el HTML de confirmación. Si llega con `Precio=2`, desaparecerá. El GET del formulario, la cookie y el enlace de modelos continuarán funcionando sin alteraciones. El alumno debe poder explicar por qué el lugar de ese código es la vista y no una nueva ruta.

El primer límite: todavía no estamos validando el negocio

En este punto el modelo `Producto` tiene propiedades ordinarias. El atributo HTML `type="number"` puede ayudar a introducir valores, pero un cliente HTTP puede enviar contenido mal formado o valores que no sean apropiados para el catálogo. También es posible que una conversión falle y que el binder registre el problema en `ModelState`. La acción inicial simplificada todavía no consulta `ModelState.IsValid` y no debe utilizarse como ejemplo de formulario listo para producción.

En 2.3 construiremos un `ProductoFormulario` específico con `[Required]`, `[StringLength]` y `[Range]`, mostraremos los errores junto a los campos y evitaremos procesar una solicitud inválida. Conviene anticipar aquí la diferencia sin explicarla dos veces:

Model binding intenta obtener valores y convertirlos a las propiedades del parámetro de acción. **Model validation** decide si los valores representan una

entrada aceptable según las reglas declaradas. **Persistencia** guarda el resultado cuando corresponde. Son operaciones relacionadas, pero no intercambiables.

Existe otra diferencia con las Web API del curso previo. Un controlador MVC que devuelve vistas no debe suponer que las respuestas 400 por validación se producen automáticamente de la misma forma que en controladores de API con `[ApiController]`. En los formularios MVC el patrón habitual será verificar `ModelState.IsValid` y devolver la misma vista con los mensajes de error para que el usuario pueda corregir los valores.

Una manera fiable de investigar fallos de formulario

En esta jornada aprenderemos a formular preguntas técnicas antes de tocar archivos al azar. Si el formulario no aparece, comprobaremos la ruta GET, la vista `Crear.cshtml` y el puerto real de Kestrel. Si aparece pero al enviarlo recibimos 405, revisaremos el atributo `[HttpPost]` y el código activado en el controlador. Si recibimos 400, investigaremos si se ha generado y remitido correctamente el token antiforgery. Si recibimos una confirmación con el precio incorrecto, compararemos el `name` del campo, el tipo de la propiedad y la cultura usada al convertir el valor.

Para una misma incidencia habrá distintas fuentes de información. La página HTML permite ver los controles y la confirmación. La pestaña **Red** del navegador muestra el método y la respuesta HTTP de cada solicitud. El código fuente del controlador explica las acciones disponibles. El catálogo permite comprobar que todavía no existe persistencia. Ninguna de estas comprobaciones requiere inventar un sistema nuevo.

El método de aprendizaje será el mismo que aprobamos en M01: formular una hipótesis, cambiar una sola responsabilidad, ejecutar, observar el resultado y explicar por qué ha cambiado. Solo después avanzaremos hacia la automatización de formularios mediante Tag Helpers.

Conexión con los puntos siguientes

El recorrido completado hasta aquí es **consulta GET → formulario Razor → POST protegido → enlace de modelos → confirmación HTML**. Hemos añadido entrada de datos al catálogo sin introducir todavía reglas de negocio ni almacenamiento. Es una evolución limitada pero real del producto anterior.

En **2.2** dejaremos de escribir manualmente cada asociación HTML y utilizaremos los Tag Helpers `asp-for`, `asp-action` y `asp-controller`. Esto hará más sencilla la correspondencia entre propiedades del modelo, etiquetas de los campos y destino de la solicitud.

En **2.3** aparecerá el problema de la entrada inválida y sus mensajes de error. En **2.4**, el flujo validado pasará de mostrar una confirmación a crear un producto persistido mediante EF Core, con SQL Server Express LocalDB en el laboratorio Windows. En **2.5** compararemos este patrón de controlador y vista con `PageModel`, `OnGet` y `OnPost` en una Razor Page mínima.

La teoría y la práctica P2.1 deben estudiarse juntas, pero no es necesario leer toda la teoría de M02 en voz alta durante la clase. La actividad esencial consiste en observar y completar un envío real; las secciones de diagnóstico ofrecen material de consulta y ampliación.

Lecturas de referencia del punto 2.1

- Microsoft Learn, Model binding en ASP.NET Core MVC: https://learn.microsoft.com/aspnet/core/mvc/models/model-binding?view=aspnetcore-8.0
- Microsoft Learn, Protección antiforgery de solicitudes: https://learn.microsoft.com/aspnet/core/security/anti-request-forgery?view=aspnetcore-8.0
- Microsoft Learn, Tag Helpers de formularios: https://learn.microsoft.com/aspnet/core/mvc/views/working-with-forms?view=aspnetcore-8.0
- Microsoft Learn, Validación de modelos en ASP.NET Core MVC: https://learn.microsoft.com/aspnet/core/mvc/models/validation?view=aspnetcore-8.0

Profundización aplicada: leer el contrato HTTP sin depender de la apariencia

Una interfaz puede parecer correcta y, sin embargo, enviar datos distintos de los que creemos. Para estudiar formularios con precisión conviene distinguir tres niveles de información. El **nivel visual** es el que observa la persona: una etiqueta «Nombre», un campo donde escribe «Rotulador» y un botón. El **nivel HTML** es el documento devuelto por el servidor: contiene atributos como `name`, `id`, `method` y `action`. El **nivel HTTP** es la solicitud real: tiene una dirección, un método, cabeceras y un cuerpo con pares de nombres y valores. MVC recibe este último nivel; no puede adivinar qué pretendíamos mostrar en pantalla.

La diferencia entre `id` y `name` merece una observación especial. `id` identifica un elemento del documento y sirve, entre otras cosas, para asociarlo con una etiqueta o localizarlo mediante JavaScript. `name` es el nombre que se utiliza cuando el navegador codifica los campos exitosos de un formulario. Si una caja de texto muestra el valor correcto pero no tiene el atributo `name` esperado, el binder no encontrará el valor bajo la clave que necesita el modelo. Renombrar únicamente el `id` no repara ese contrato. Esta distinción será muy útil cuando en 2.2 dejemos que `asp-for` genere ambos atributos.

La codificación de formularios estándar, `application/x-www-form-urlencoded`, representa los datos como pares nombre–valor. Aunque el navegador presente un control de tipo número, lo que llega por HTTP sigue siendo una representación textual. ASP.NET Core debe convertirla al tipo de la propiedad C#. Esto explica por qué hay dos familias de errores que a primera vista pueden parecer iguales: ausencia de un valor, porque el campo no viajó o está vacío, y valor no convertible, porque el texto enviado no puede interpretarse como el tipo de destino. El binder interviene en ambos casos, pero el diagnóstico no debe confundirlos.

También debemos separar la semántica de GET y POST de los efectos de la aplicación. HTTP no convierte automáticamente cualquier GET en una consulta segura ni cualquier POST en una inserción SQL: es nuestro código el que decide qué operaciones realiza. Utilizar GET para solicitar un formulario sigue siendo apropiado porque leerlo no debe modificar el estado. Utilizar POST para enviar una operación que puede producir cambios hace explícita la intención y habilita mecanismos como la comprobación antiforgery. Un formulario que recibe POST y presenta una confirmación, como en 2.1, todavía no demuestra persistencia; un formulario que recibe POST y ejecuta `SaveChangesAsync`, como en 2.4, sí puede producirla.

Imaginemos tres incidencias distintas en la misma pantalla. En la primera, el formulario envía a una URL equivocada y obtenemos 404: el problema es el destino. En la segunda, la URL es correcta pero solo existe una acción GET y recibimos 405: el problema es la combinación de método y endpoint. En la tercera, la acción POST existe y está protegida, pero construimos una petición sin token y obtenemos 400: el problema es la verificación de seguridad. Cambiar una anotación `[Required]` no resuelve ninguna de esas incidencias. La habilidad profesional consiste en relacionar cada síntoma con la fase exacta del recorrido, no en ir modificando archivos al azar.

El antiforgery se comprende mejor como comprobación de procedencia de una solicitud que como una propiedad «mágica» del formulario. La cookie identifica

contexto, el campo oculto lleva un token y el servidor compara la información necesaria antes de permitir ejecutar la acción protegida. Por ello, copiar solo el valor del campo oculto entre navegadores o sesiones no equivale a reproducir el envío legítimo. Tampoco es una defensa general frente a ataques de inyección, validación deficiente o permisos incorrectos: esas preocupaciones requieren controles diferentes.

Actividad de reflexión opcional. Observa el HTML de `Crear.cshtml` y responde, justificando cada decisión, qué componentes intervienen si el usuario cambia el texto de una etiqueta, si cambia el atributo `name` de un campo, si cambia el método del formulario y si cambia la acción de destino. En el primer caso puede cambiar solo la experiencia visual; en los otros tres puede romperse el contrato recibido por MVC. La respuesta razonada importa más que memorizar una tabla de atributos: te prepara para diagnosticar el comportamiento de formularios que no has desarrollado tú mismo.

2.2. Tag Helpers: de los atributos HTML escritos a mano al enlace tipado con MVC

La misma aplicación, un nuevo problema de mantenimiento

En el capítulo anterior conseguimos que un formulario enviara el nombre y el precio de un producto al servidor. El controlador disponía de dos acciones llamadas Crear: una GET para mostrar la página y otra POST para procesar los campos. También comprobamos que un token antiforgery permite distinguir un envío normal desde el navegador de una petición que intenta omitir esa protección. Ahora vamos a conservar esas responsabilidades, pero mejorar la **manera de describir el formulario y sus enlaces dentro de Razor**.

Observemos cómo era la primera solución. En el HTML de la vista escribíamos explícitamente el destino de la petición, los nombres de los campos, los identificadores y las asociaciones entre etiquetas y entradas. No había nada intrínsecamente incorrecto en ello: el navegador recibió controles HTML válidos. El problema aparece cuando el programa crece. Si la acción Crear se renombra o una propiedad del modelo cambia de nombre, las cadenas escritas manualmente no expresan esa dependencia con tanta claridad. Un formulario puede seguir dibujándose correctamente y, sin embargo, enviar campos con nombres que ya no coincidan con las propiedades del modelo.

En M01 vimos una situación equivalente al pasar de enlaces con rutas escritas a mano a enlaces generados mediante **Anchor Tag Helper**. El sistema de routing conocía las acciones MVC y sus parámetros, mientras que un enlace literal dependía de que nosotros recordásemos sus segmentos. El punto 2.2 amplía esa misma idea a formularios, etiquetas, campos y botones. No introduce otra arquitectura, no altera la base de datos —todavía inexistente en P2.2— y no sustituye HTML por un lenguaje de interfaz nuevo.

Un Tag Helper es una funcionalidad de ASP.NET Core que participa en la generación de HTML a partir de un elemento Razor dotado de atributos especiales. Escribimos marcado que conserva la estructura habitual de HTML y añadimos atributos como asp-for o asp-action. Al procesar la vista, el servidor interpreta esos atributos y produce elementos HTML ordinarios para el navegador. No es un componente Blazor interactivo; los componentes Razor se estudiarán en M03 y tienen otro modelo de ejecución.

No debemos interpretar el cambio como una «magia» que elimina la necesidad de comprender HTTP o HTML. De hecho, el ejercicio más útil consiste en comparar el código fuente Razor con el **HTML realmente devuelto**. Esa comparación hace visible la responsabilidad del Tag Helper y ayuda a detectar errores cuando no se reconoce una directiva.

Dos documentos distintos: la vista Razor y el HTML recibido

La vista es un archivo CSHTML que puede contener sintaxis de Razor y Tag Helpers. Por ejemplo, el atributo especial de la siguiente etiqueta no pertenece al HTML estándar:

```
<label asp-for="Nombre"></label>
```

El navegador no debería recibir un atributo asp-for sin procesar. El Label Tag Helper lo transforma, para la propiedad Nombre del modelo, en un resultado conceptualmente equivalente a:

```
<label for="Nombre">Nombre</label>
```

Ahora sí estamos ante HTML ordinario: la etiqueta señala un control identificado por Nombre. La expresión asp-for no se ha ejecutado en JavaScript ni aparece como un «comando» para el navegador. La transformación se ha realizado **en el servidor durante la representación de la vista**.

Compárese con lo que ocurría en P2.1:

```
<label for="Nombre">Nombre</label>
<input id="Nombre" name="Nombre" value="@Model.Nombre" />
```

Ambas representaciones pueden producir un resultado visual indistinguible. El valor del nuevo enfoque no se mide contando píxeles diferentes: se mide observando que la vista describe una asociación tipada, que la infraestructura puede producir atributos coherentes con el modelo y que dejamos de repetir tantas cadenas manuales.

Conocer la diferencia fuente/resultado resulta imprescindible para depurar. Si alguien dice «he añadido asp-for y no se ve ningún cambio», quizá no exista un problema: puede que el resultado HTML correcto de la nueva etiqueta sea casi idéntico al anterior. Lo que interesa comprobar es el vínculo entre la propiedad C# y

los atributos finales, la continuidad del model binding y las comprobaciones de entrada que ya aprendimos.

Dónde se habilitan los Tag Helpers

Antes de modificar un formulario conviene averiguar por qué los atributos especiales funcionan en nuestro proyecto. El archivo **Views/_ViewImports.cshtml** contiene:

```
@using CursoWeb.Mvc
@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers
```

La directiva addTagHelper hace disponibles los Tag Helpers integrados de ASP.NET Core MVC para las vistas que quedan dentro de ese ámbito de importación. No es un controlador y no constituye una llamada HTTP. Tampoco instala Entity Framework Core, añade propiedades a Producto ni define por sí mismo rutas de negocio.

La posición del archivo es importante. Situado en Views, su contenido se aplica a las vistas de ese directorio y sus subdirectorios, como Views/Productos/Crear.cshtml. Esta decisión evita repetir la directiva en cada CSHTML. El alumnado ya conocía el archivo desde M01, porque allí se utilizaban Tag Helpers de parciales y enlaces; ahora va a comprender su función de forma más explícita.

El prefijo asp- puede inducir a pensar que todo atributo que comienza así es una propiedad HTML. No lo es: ciertos atributos se interpretan específicamente durante el procesamiento de Razor, por un Tag Helper que se ha registrado. Si la directiva correspondiente no está habilitada o el elemento está fuera del ámbito, esos atributos podrían terminar sin procesarse en el HTML que recibe el navegador. Ésta es una de las primeras hipótesis que conviene comprobar cuando una vista «muestra asp-for» en lugar del id y name esperados.

No desactivaremos _ViewImports.cshtml para realizar la práctica. Ya existen otras vistas del proyecto que dependen de Tag Helpers, especialmente la vista parcial del catálogo. El objetivo de P2.2 no es romper la funcionalidad anterior para observar un fallo artificial, sino ampliar su uso a un formulario nuevo.

La documentación oficial de Microsoft sobre Tag Helpers y _ViewImports explica tanto el registro mediante addTagHelper como la diferencia con los antiguos HTML Helpers. El archivo del curso es un ejemplo deliberadamente pequeño de ese mecanismo.

Primer cambio: Form Tag Helper y la dirección de destino

El formulario de P2.1 expresaba su destino con un atributo HTML literal:

```
<form method="post" action="/Productos/Crear">
    @Html.AntiForgeryToken()
    <!-- campos de entrada -->
</form>
```

Con el **Form Tag Helper** podemos describir el controlador y la acción en lugar de escribir toda la ruta en una única cadena:

```
<form asp-controller="Productos" asp-action="Crear" method="post">
    <!-- campos de entrada -->
</form>
```

Aquí asp-controller identifica el controlador lógico Productos y asp-action identifica la acción Crear. El atributo method continúa siendo HTML estándar: determina que el formulario se envía por POST. El Form Tag Helper utiliza la configuración de routing de la aplicación para generar el atributo action final. En nuestro proyecto y con la ruta convencional disponible, el navegador recibe una apertura equivalente a:

```
<form method="post" action="/Productos/Crear">
```

Es tentador concluir que no hemos ganado nada porque la dirección generada es la misma. Sin embargo, ahora el origen de esa dirección está relacionado con los elementos de MVC, en lugar de ser una ruta literal dispersa por una vista. Cuando en otra práctica estudiemos rutas diferentes o se cambien convenciones, el resultado podrá evolucionar con ellas. No significa que cualquier cambio de nombre quede arreglado automáticamente, ni que el compilador garantice siempre la existencia de una acción: habrá que seguir comprobando la ruta final.

El proyecto P2.2 empieza conservando activo el formulario HTML de P2.1. Dentro de Crear.cshtml se entrega, comentada, la alternativa de Form Tag Helper. El alumno sustituirá la apertura original, sin duplicar formularios. La etiqueta de cierre permanece, porque no se está creando otra estructura HTML: es el mismo formulario descrito con otros atributos.

El token antiforgery: continuidad funcional y una precaución importante

Recordemos que P2.1 utilizaba Html.AntiForgeryToken() para escribir explícitamente el campo oculto. En P2.2 emplearemos el Form Tag Helper integrado, con method="post", asp-controller y asp-action. En ese contexto ASP.NET Core **puede generar automáticamente el token antiforgery**. En este ejemplo concreto, con un formulario POST generado por el Tag Helper, esa generación forma parte del resultado esperado.

Por esa razón, al reemplazar la etiqueta original eliminaremos la llamada explícita a Html.AntiForgeryToken(). No es una invitación a retirar la protección, sino a evitar introducir dos mecanismos de escritura del mismo campo cuando el propio Form Tag Helper se encarga de generar el token. **El controlador conserva el atributo ValidateAntiForgeryToken** y el navegador sigue enviando su información antiforgery.

La comprobación se realiza sobre el HTML final. Debemos encontrar **un campo oculto** con el nombre __RequestVerificationToken en el formulario que se genera. Si el alumno elimina la llamada explícita pero la etiqueta sigue siendo HTML sin intervención de Form Tag Helper, el token podría desaparecer; el POST empezaría a responder 400. Esa incidencia es precisamente el tipo de síntoma que permite relacionar una decisión de maquetación de formulario con una consecuencia HTTP.

No confundiremos la generación del token con la autorización de un usuario. El token antiforgery ayuda a defender frente a falsificación de solicitudes entre sitios, pero no determina quién puede crear productos y no valida las reglas del precio. De nuevo, tenemos tres preguntas independientes: cómo se genera la petición, si su origen cumple las comprobaciones previstas y si los datos del negocio son aceptables.

La documentación oficial de Form Tag Helper explica la generación del action y del token oculto. La comprobación práctica consistirá en cargar una página real, inspeccionarla y enviar el formulario, no en copiar tokens desde una captura.

Segundo cambio: Label Tag Helper y asociación de etiquetas

En HTML ordinario se asocian un label y un input escribiendo sus atributos explícitamente:

```
<label for="Nombre">Nombre</label>
<input id="Nombre" name="Nombre" />
```

Esta asociación tiene un valor funcional y de accesibilidad. Al seleccionar la etiqueta con el puntero, el navegador puede enfocar el control correspondiente. También aporta información útil a las tecnologías de asistencia. El texto Nombre no debe estar separado de su campo por razones puramente estéticas.

Con el Label Tag Helper escribimos:

```
<label asp-for="Nombre"></label>
```

El resultado es conceptualmente equivalente al HTML original. El nombre de la propiedad se interpreta en el contexto de la directiva @model de la vista, que en P2.2 sigue siendo CursoWeb.Mvc.Models.Producto. El Label Tag Helper no procesa la cadena enviada por el usuario: utiliza **metadatos del modelo de la vista** para decidir a qué control corresponde la etiqueta.

Esta diferencia será aún más útil en el punto 2.3, cuando aparezcan anotaciones como Display o Required en modelos de entrada. Por ejemplo, una propiedad técnica podría tener un nombre de presentación distinto para el usuario. El Tag Helper está preparado para utilizar esa información, mientras que un literal escrito en HTML debe revisarse manualmente en cada vista afectada.

En P2.2 las etiquetas de Nombre y Precio se transforman por separado. Es importante que el alumno no cambie a la vez etiquetas, campos y destino del formulario sin observar los estados intermedios. Primero deberá comprobar que el atributo for de ambas etiquetas se genera correctamente y que los campos todavía mantienen sus valores y nombres originales. Si algo deja de funcionar, el cambio responsable estará acotado.

No hay una diferencia entre «label con Tag Helper» y «label sin HTML»

El resultado sigue siendo un elemento label de HTML. Un Tag Helper no inventa una interfaz visual alternativa. En esta actividad, la etiqueta no aparecerá en una pantalla distinta ni mostrará necesariamente un texto espectacular. El criterio observable será que, al inspeccionar el HTML, for apunta al id del input y el atributo asp-for ya no aparece sin procesar.

Esa forma de evaluar protege contra una mala costumbre pedagógica: dar por terminado un ejercicio porque «la página se ve igual». El alumnado ha de demostrar que sabe por qué es apropiado el elemento generado y cómo se conecta con el modelo.

Tercer cambio: Input Tag Helper y generación de nombre, id y valor

El Input Tag Helper es uno de los mecanismos que más fácilmente conecta los conocimientos de M01 con el nuevo model binding de M02. Hasta ahora, el formulario tenía entradas explícitas:

```
<input id="Nombre" name="Nombre" value="@Model.Nombre" />
<input id="Precio" name="Precio" type="number" step="0.01"

value="@Model.Precio.ToString(System.Globalization.CultureInfo.InvariantCulture)" />
```

La primera entrada utiliza el nombre de campo Nombre y la segunda, Precio. Cuando el navegador construye el POST, esas claves indican al binder dónde encontrar cada valor. El atributo value de cada entrada representa el valor inicial escrito al renderizar la vista.

La forma con Input Tag Helper es más breve:

```
<input asp-for="Nombre" />
<input asp-for="Precio" type="number" step="0.01" />
```

En los dos casos asp-for apunta a una propiedad del modelo tipado. El resultado generado incluye atributos id y name apropiados para el enlace de modelos, y el valor inicial procede del modelo. Para Nombre se obtendrá un control textual. Para Precio conservamos explícitamente type="number" y step="0.01", porque estamos explicando cómo se usa un control numérico concreto y no queremos convertir una inferencia de tipo HTML en el objetivo de la práctica.

Es esencial recordar que **el atributo asp-for no valida por sí solo los valores del negocio**. Que el navegador proporcione un control de tipo number no significa que el servidor deba aceptar cualquier cantidad, ni que un cliente HTTP no pueda enviar campos con valores arbitrarios. En P2.3 utilizaremos un modelo de entrada y reglas declarativas de validación.

El alumno debe comparar el nombre y el identificador del control generado con los de la versión anterior. Si el formulario deja de construir un Producto cuyos campos coincidan con el valor enviado, no basta con cambiar el código del controlador sin investigar: hay que inspeccionar el atributo name del HTML realmente enviado.

La cultura del precio: qué podemos y qué no podemos concluir

El formulario de P2.1 convertía inicialmente el valor decimal a una cadena utilizando InvariantCulture. Con Input Tag Helper es el framework quien construye parte de la representación a partir del tipo y de los metadatos de la propiedad. Esta automatización no constituye, por sí sola, una solución universal para diferentes culturas, separadores decimales y reglas de validación.

En las comprobaciones principales del curso enviaremos precios enteros, como 7 y 2, de modo que podamos observar el model binding sin introducir simultáneamente la globalización. Si necesitásemos ofrecer un formulario público para diferentes idiomas y regiones, tendríamos que establecer y probar una política de cultura de solicitud coherente. La explicación completa de ese escenario excede este punto de seis horas.

No debemos recomendar que el alumno «arregle» cualquier error de precio cambiando todas las propiedades a string. Precio representa una cantidad y continuará siendo decimal en el dominio. Lo que cambiaremos cuando corresponda es el contrato de entrada y las reglas que permiten informar correctamente al usuario de un error de conversión.

Cuarto cambio: Anchor Tag Helper para volver al catálogo

La navegación también forma parte de la experiencia de un formulario. El usuario debe poder abandonar la pantalla de alta sin introducir datos o volver al catálogo después de revisar una confirmación. En la versión HTML original escribimos:

```
<a href="/Productos">Volver al catálogo</a>
```

Con Anchor Tag Helper podemos indicar el destino mediante los elementos MVC:

```
<a asp-controller="Productos" asp-action="Index">Volver al catálogo</a>
```

El HTML generado incluirá un atributo href que el navegador entiende. El Tag Helper no envía la petición en nombre del usuario: únicamente genera un enlace. Cuando el usuario pulse ese enlace, el navegador iniciará una nueva solicitud GET al destino representado.

No confundiremos el atributo asp-action de una etiqueta a con el atributo asp-action de un formulario. Ambos utilizan los datos de routing, pero el primero participa en la generación de un enlace de navegación, mientras que el segundo determina el

destino al enviar el formulario. El método HTTP de la navegación ordinaria sigue siendo GET. En cambio, el formulario de creación mantiene method="post".

Esta diferencia es un ejemplo de por qué no conviene estudiar Tag Helpers como una colección de palabras clave sin contexto. En la práctica se utilizarán tres tipos de elementos: form, input/label y a. Cada uno produce una parte distinta del HTML final y participa en una operación distinta del usuario.

Un paso adicional: enlaces cuyo identificador procede de un producto

El catálogo contiene tres productos con identificadores 1, 2 y 3. Sus enlaces originales ya funcionan:

```
<a class="enlace-detalle" href="/Productos/Detalle/@producto.Id">Ver detalle</a>
```

La expresión Razor produce la parte variable de la cadena. Para un producto con Id igual a 2, la URL final debe permitir visitar la ficha de Lápiz. Esta solución es correcta para el ejemplo, pero sigue escribiendo de forma literal la parte fija de la ruta.

Podemos expresar el mismo enlace de acuerdo con el routing de MVC:

```
<a class="enlace-detalle"
   asp-controller="Productos"
   asp-action="Detalle"
   asp-route-id="@producto.Id">Ver detalle</a>
```

El atributo asp-route-id aporta el valor de la ruta llamado id. En nuestro proyecto, la ruta convencional incluye el segmento opcional {id?}; por eso un valor como 2 puede incorporarse a la URL final de detalle. El nombre que aparece tras asp-route- debe corresponder al valor de ruta que quiere suministrarse; no es una propiedad arbitraria del navegador.

La prueba correcta no consiste en generar un enlace fijo para Lápiz. El listado debe contener **tres enlaces diferentes** y cada uno debe llegar a su ficha. Si la acción se trasladara a otra ruta convencional o se ajustase su configuración, habría que revisar la URL producida, pero el objetivo del Tag Helper es describir el destino utilizando los conceptos de MVC y sus valores de ruta.

Este es el **reto de P2.2**. El alumno deberá activar una alternativa comentada dentro del foreach del listado y retirar el enlace literal que sustituye. Así se evita

terminar con dos enlaces de detalle por producto. No se modifica el controlador ni se guarda información en una base de datos.

El caso especial del producto recién enviado

En P2.2, el modelo Producto que llega desde el formulario puede tener Id igual a cero, porque el formulario no envía un identificador y todavía no se guarda nada. Por eso **no vamos a crear automáticamente en la confirmación un enlace de detalle utilizando Model.Id**: hacerlo llevaría a una dirección que no identifica necesariamente un producto guardado.

Esta observación conecta directamente con el problema que resolverá el CRUD de P2.4. Una vez guardado un producto en la base de datos, tendrá un identificador persistente y la aplicación podrá redirigir a su detalle. Hasta entonces, el botón correcto de la confirmación es «Enviar otro formulario» o «Volver al catálogo», no «Ver el producto recién creado» como si existiera en almacenamiento.

Es una distinción pequeña pero pedagógicamente importante: **un Tag Helper puede generar una URL sintácticamente válida y, aun así, esa URL puede no representar un recurso existente**. La infraestructura de routing no sustituye las reglas de negocio.

De los HTML Helpers a los Tag Helpers: una decisión, no una competición

En P2.1 utilizamos @Html.AntiForgeryToken(), un HTML Helper. En P2.2 utilizamos atributos de Tag Helpers. Ambos mecanismos pueden convivir en una misma aplicación; la plataforma no obliga a convertir de inmediato todos los HTML Helpers a otra sintaxis.

La elección depende de qué resultado queremos producir y de cómo deseamos expresar el código Razor. El Form Tag Helper reduce el trabajo de escribir rutas y, en el caso apropiado, incorpora la generación del token. El Label y el Input Tag Helper aprovechan los metadatos de las propiedades y mantienen coherencia entre etiquetas y controles. El Anchor Tag Helper permite expresar enlaces en términos de controlador, acción y valores de ruta.

Un error sería enseñar que los Tag Helpers impiden todas las equivocaciones o que generan interfaces de usuario seguras sin más comprobaciones. Son una ayuda para describir elementos HTML a partir del contexto del servidor. La aplicación necesita seguir considerando la validación, las autorizaciones cuando corresponda,

la accesibilidad, la representación cultural de los datos y el mantenimiento de sus rutas.

En esta jornada evitaremos introducir Tag Helpers personalizados. Su creación es una técnica útil para otras aplicaciones, pero excede el alcance reducido del curso. La prioridad es trabajar con los Tag Helpers incorporados a ASP.NET Core MVC y entender sus efectos en un formulario real.

Diagnóstico razonado de un formulario convertido

Imaginemos que hemos sustituido la apertura del formulario, ambos labels, los dos inputs y el enlace de vuelta al catálogo. La página se sigue viendo igual. ¿Cómo demostramos que hemos hecho un buen trabajo?

En primer lugar, el HTML generado no debe contener asp-controller, asp-action ni asp-for como atributos sin procesar en los elementos transformados. Si aparecen literalmente en el DOM, revisaremos el archivo _ViewImports.cshtml y la ubicación de la vista. No cambiaremos de forma aleatoria las acciones del controlador.

En segundo lugar, la etiqueta form debe disponer de un action apropiado y mantener method="post". El atributo action es el que comprende el navegador; asp-action es la instrucción utilizada para generarlo. El formulario no ha cambiado su finalidad por haber cambiado su sintaxis Razor.

En tercer lugar, hay que comprobar que cada label dispone de un atributo for que coincide con el id del campo correspondiente. La comprobación aporta significado y accesibilidad, no solamente un resultado estético.

En cuarto lugar, los inputs deben conservar name="Nombre" y name="Precio" en el HTML generado. Si se cambia accidentalmente una de esas claves, el enlace de modelos puede recibir valores vacíos o predeterminados, aunque el usuario haya escrito algo en pantalla. El HTML que sale del servidor y el POST que llega al controlador son las dos caras del contrato.

Por último, hay que examinar el token: debe existir un campo oculto __RequestVerificationToken que se envía al servidor cuando corresponde, y la acción POST debe seguir comprobándolo. Una petición sin token válido continúa obteniendo 400, mientras que una solicitud enviada correctamente desde la página puede mostrar la confirmación.

No se necesita una pantalla nueva para cada comprobación. Basta con **seguir el mismo recorrido del usuario y examinar evidencias concretas**. Este criterio hace que las prácticas de P2.1 y P2.2 sean comparables y que el alumnado no pierda lo que ya había conseguido.

Errores que parecen similares, pero tienen causas diferentes

Si el formulario responde con 405, la ruta de la acción POST puede no estar disponible. La conversión a Tag Helpers no crea automáticamente una acción que no exista.

Si responde con 400, hay que considerar el token antiforgery y otros problemas de solicitud. Después de haber cambiado el form, un error común sería eliminar la llamada manual al token sin haber activado realmente el Form Tag Helper: el HTML dejaría de contener el campo necesario.

Si el POST devuelve confirmación pero con Nombre vacío, podemos investigar el atributo name generado en el input, sin cambiar de inmediato el código que representa Confirmacion.cshtml.

Si el enlace de regreso al catálogo lleva a una dirección incorrecta, debemos comparar el href generado y las reglas de routing. La apariencia de un enlace subrayado no demuestra que el destino sea correcto.

Si se representan dos enlaces de detalle por producto, probablemente el alumno ha **añadido** la variante del reto sin retirar el enlace original. Éste es un problema de duplicación del marcado, no del motor MVC.

La forma de resolver estas incidencias consiste en formular una hipótesis, encontrar la capa responsable y cambiar el archivo correcto. El diagnóstico no se basa en copiar una configuración desconocida desde Internet ni en eliminar protecciones para hacer que el navegador avance.

El límite de P2.2 y el puente hacia P2.3

Ya podemos describir la entrada y la navegación utilizando los servicios que MVC proporciona durante la representación de Razor. Hemos mejorado el formulario y sus enlaces **sin alterar el contrato funcional**: el navegador sigue enviando Nombre y Precio; el controlador sigue recibiendo Producto; antiforgery sigue comprobándose; y la confirmación sigue mostrando lo que llegó.

La pregunta que todavía no hemos contestado es si todos esos valores deben aceptarse. ¿Qué hacemos si alguien deja Nombre vacío, introduce un precio negativo o supera una longitud admitida por la aplicación? Ni asp-for ni la presentación de un campo con type="number" ofrecen por sí solos la respuesta completa. El siguiente capítulo construirá un modelo de entrada específico, utilizará anotaciones como Required, StringLength y Range y observará el estado de ModelState.

En P2.3 añadiremos mensajes visibles junto a los campos y comprobaremos qué ocurre cuando el formulario debe volverse a representar sin perder los valores enviados. Ese paso ya no será una transformación puramente estructural del HTML: introducirá decisiones de aceptación de datos en el servidor y un mecanismo para informar al usuario de los errores.

Después, en 2.4, usaremos la base SQL Server Express LocalDB con Entity Framework Core que el alumnado ya conoce. La entrada validada pasará a las operaciones reales de creación, edición y eliminación. Finalmente, en 2.5 compararemos la coordinación mediante controlador y vista con los métodos OnGet y OnPost de una Razor Page mínima alojada en la misma aplicación.

Fuentes y documentación relacionada

- Microsoft Learn, Tag Helpers en formularios de ASP.NET Core.
- Microsoft Learn, Introducción a los Tag Helpers.
- Microsoft Learn, Anchor Tag Helper.
- Microsoft Learn, Model binding.

Profundización aplicada: Tag Helpers como traducción controlada, no como HTML oculto

Una de las dificultades al comenzar con Tag Helpers consiste en creer que reemplazan por completo al HTML. En realidad, trabajan **antes** de que el documento llegue al navegador. El archivo `.cshtml` contiene instrucciones reconocibles para Razor y extensiones de atributos como `asp-for` y `asp-action`; la respuesta HTTP contiene etiquetas HTML ordinarias. Esta transformación debe resultar observable, no misteriosa. Si inspeccionamos el código fuente generado, podemos analizar qué `action` se calculó, qué `name` y `id` se produjeron para cada entrada y cómo se construyeron los enlaces.

Pensemos en un cambio frecuente durante el mantenimiento: la acción de creación deja de llamarse `Crear` y pasa a tener otra ruta pública. En el HTML escrito manualmente, una cadena fija puede quedar desactualizada si nadie la revisa. El Form Tag Helper utiliza información del enrutamiento y los nombres de controlador y acción para generar la dirección. Esta ventaja no significa que el compilador garantice todos los destinos: si escribimos un nombre de acción incorrecto como cadena en `asp-action`, podremos seguir produciendo una URL no deseada. Lo que ganamos es **integración con la infraestructura de rutas** y un lugar claro para localizar el problema.

El Input Tag Helper también necesita un contrato. Con `asp-for="Nombre"`, Razor conoce el tipo de modelo y la expresión a la que se refiere el campo. A partir de ahí genera atributos y, cuando existen metadatos de validación, puede generar información que la validación del navegador interprete. No convierte por sí mismo texto inválido en un `decimal` válido: después del envío sigue siendo necesaria la conversión en el servidor. No vuelve segura una operación por el mero hecho de usar una etiqueta más expresiva. La fiabilidad procede de combinar marcado, binding, validación y protección del endpoint.

En las vistas parciales aparece otro aspecto importante: **contexto**. Cuando una parcial recibe un `Producto` individual, un enlace con `asp-route-id="@Model.Id"` representa el identificador del producto que está dibujando. Si reutilizamos la misma parcial en un detalle y en una lista, el enlace debe seguir relacionándose con el modelo que recibe. Sin ese contrato, podríamos producir una colección de tarjetas bonitas que apuntan todas al mismo recurso. Los Tag Helpers no corrigen una parcial que recibe un modelo equivocado; hacen más explícita la construcción del enlace, pero necesitan datos correctos.

Conviene prestar atención al HTML resultante en varios casos: un nombre con espacios, un texto con caracteres especiales y un identificador numérico insertado en la ruta. Razor codifica normalmente los valores representados en HTML y la infraestructura de routing se ocupa de generar destinos; eso no significa que debamos concatenar cadenas procedentes del usuario para fabricar etiquetas o atributos completos. Mantener los datos como datos, y no como fragmentos de marcado, facilita que el framework realice la codificación apropiada.

Otra confusión habitual es atribuir al Tag Helper el comportamiento de JavaScript. `asp-validation-for` puede producir una ubicación donde representar un mensaje y atributos relacionados con validación. Sin embargo, la ejecución de validación

cliente depende de los scripts instalados y cargados, y la aceptación final en el servidor depende de `ModelState`. Si retiramos los scripts, podemos perder inmediatez al informar de un error, pero no debemos perder el rechazo de entradas inválidas en la acción. Por tanto, el comportamiento observado en la pantalla no puede considerarse prueba suficiente de que el servidor valida correctamente.

Diagnóstico de transferencia. Si el formulario funciona sin estilos pero deja de enviar después de convertirlo a Tag Helpers, evita revisar primero CSS. Inspecciona la etiqueta `form` generada, compara su `action` con la ruta MVC registrada, verifica `method="post"` y comprueba que viaja el token. Si el botón envía pero `Nombre` llega vacío, inspecciona el atributo `name` generado y la propiedad del modelo de entrada. Si un enlace de tarjeta apunta al Id equivocado, examina qué objeto se pasó a la parcial y qué valor se utilizó al construir `asp-route-id`. Cada diagnóstico conduce a un archivo distinto.

2.3. Validación de formularios con DataAnnotations y ModelState

La nueva pregunta: no basta con recibir datos

Al terminar P2.2, el catálogo tiene un formulario bastante más expresivo que el de P2.1. Podemos declarar su destino mediante `asp-controller` y `asp-action`, asociar etiquetas con propiedades mediante `asp-for`, y generar los campos de entrada de forma coherente con el modelo tipado. La página se representa en el servidor y el navegador puede remitir sus valores mediante POST. También conservamos la protección antiforgery del primer ejercicio.

Esa mejora no responde, sin embargo, a una pregunta fundamental: **¿qué hacemos con los datos que no representan un producto aceptable?** Un campo vacío, un precio negativo, un texto excesivamente largo o una cadena que no pueda convertirse en un decimal pueden llegar al servidor de distintas maneras. El atributo `type="number"` facilita la edición en el navegador, pero no constituye una defensa suficiente: se pueden construir peticiones HTTP sin interactuar con ese control, el JavaScript puede estar deshabilitado y otros clientes pueden enviar directamente pares de nombre y valor.

Para comprender la diferencia, pensemos en dos situaciones. Una persona escribe Nombre = «Carpeta» y Precio = 8. Un objeto `ProductoFormulario` con esos valores puede cumplir nuestras reglas y convertirse en un producto. Otra escribe Nombre = «X» y Precio = -5. El navegador puede transportar esos datos, pero la aplicación debe saber que no cumplen el contrato de negocio. **Recibir información y aceptarla son operaciones distintas.** Es precisamente la responsabilidad que añadiremos en este capítulo.

Nuestro aprendizaje seguirá una progresión deliberada. Primero entenderemos por qué el modelo que recibe el formulario no tiene por qué ser la misma clase que la entidad del catálogo. Después definiremos sus reglas con atributos declarativos de C#. A continuación observaremos qué registra `ModelState` cuando se produce un error, cómo devuelve el controlador la misma página y de qué manera las vistas Razor muestran mensajes junto a los campos. Finalmente diferenciaremos la validación en el servidor de la validación anticipada que puede realizar el navegador cuando se habilitan las bibliotecas correspondientes.

El objetivo no es impartir otra vez Entity Framework Core: los alumnos ya conocen esa herramienta, pero **P2.3 todavía no persiste** datos. Reservamos la operación de almacenamiento y los identificadores reales para el CRUD de P2.4, cuando podamos confiar en un contrato de entrada validado.

Por qué separar Producto y ProductoFormulario

En M01, `Producto` era un objeto de dominio sencillo:

```
public sealed class Producto
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public decimal Precio { get; set; }
}
```

Servía para representar los elementos de la colección del catálogo, seleccionar una ficha y obtener los valores mostrados en Razor. En P2.1 y P2.2 también lo usamos como parámetro de entrada por simplicidad. Ese diseño permitió observar el mecanismo de model binding, pero ahora conviene evolucionarlo.

Un formulario de **alta** de producto necesita recibir un nombre y un precio; normalmente no debería permitir al usuario elegir el identificador persistente que asignará el sistema. Si utilizamos la entidad directamente como contrato de entrada, corremos el riesgo de aceptar propiedades que el usuario no debería controlar, problema conocido como **sobrepublicación o overposting**. La importancia de este aspecto crece cuando las entidades añaden campos como Activo, PropietarioId o PrecioAutorizado, que no deben quedar bajo control de cualquier petición HTML.

El proyecto introduce una clase diferente:

```
public sealed class ProductoFormulario
{
    public string? Nombre { get; set; }
    public decimal? Precio { get; set; }
}
```

El nuevo tipo tiene únicamente las propiedades necesarias para el formulario. No expone `Id`. Al mismo tiempo, ambas propiedades se declaran anulables. Esa decisión es relevante para los experimentos: **un campo que no llega y un valor numérico explícito no son la misma cosa**. Con `decimal?` podemos distinguir un

precio ausente (`null`) del número cero (`0`) y del número siete (`7`). En una propiedad no anulable `decimal`, un valor ausente puede acabar representado por el valor predeterminado cero, lo que dificulta explicar por qué ha fallado una regla.

La propiedad `Nombre` es `string?` porque el formulario puede llegar sin ese campo. Que la propiedad permita almacenar `null` en C# no implica que el negocio lo acepte. Esa diferencia entre **posibilidad técnica del tipo** y **restricción del dominio** es precisamente lo que hacen visible las DataAnnotations.

Una vez validada la entrada, la acción puede crear un objeto de dominio:

```
var producto = new Producto
{
    Id = 4, // Identificador de demostración: P2.3 aún no guarda.
    Nombre = entrada.Nombre!,
    Precio = entrada.Precio!.Value
};
return View("Confirmacion", producto);
```

El número 4 no es un identificador asignado por SQL Server; pertenece solamente a esta demostración temporal. En P2.4 lo eliminaremos del flujo de creación y dejaremos que la persistencia proporcione un identificador real. El operador `!` expresa al compilador que confiamos en la nulabilidad después de comprobar la entrada; no constituye una validación por sí mismo.

El modelo de entrada no es una segunda aplicación

No estamos duplicando el catálogo. Conservamos las mismas rutas, el mismo controlador y la misma vista de confirmación; lo que cambia es el **tipo aceptado por el POST**. La vista `Crear.cshtml` trabaja con `ProductoFormulario`, mientras `Confirmacion.cshtml` recibe `Producto`. La transformación entre ambos tipos está situada en el controlador, donde resulta explícita y fácil de explicar.

Esta separación se puede trasladar a sistemas mayores sin introducir complejidad innecesaria. En el curso no añadiremos una biblioteca de mapeo ni una capa de servicios adicional solo para copiar dos propiedades. El alumnado conoce C# y puede comprender la asignación directa. La prioridad es identificar qué información entra desde HTTP y qué entidad representa el catálogo.

DataAnnotations: reglas declaradas junto al contrato de entrada

Las **DataAnnotations** son atributos de .NET que expresan, entre otros usos, restricciones de validación sobre propiedades de un modelo. El espacio de nombres principal es `System.ComponentModel.DataAnnotations`. En lugar de escribir en cada acción una serie de condiciones repetidas para el nombre y el precio, definiremos las reglas relevantes una sola vez en `ProductoFormulario`.

El ejercicio utiliza tres tipos de anotaciones. `[Required]` indica que el campo es obligatorio; `[StringLength]` limita la longitud del nombre, con mínimo y máximo; y `[Range]` expresa el intervalo admitido para el precio. El proyecto incluye las líneas correspondientes **comentadas**, de modo que el alumno pueda activarlas y observar qué errores aparecen.

```
using System.ComponentModel.DataAnnotations;

public sealed class ProductoFormulario
{
    [Required(ErrorMessage = "Indica el nombre.")]
    [StringLength(60, MinimumLength = 2,
        ErrorMessage = "El nombre debe tener entre 2 y 60 caracteres.")]
    public string? Nombre { get; set; }

    [Required(ErrorMessage = "Indica el precio.")]
    [Range(typeof(decimal), "0.01", "9999.99",
        ParseLimitsInInvariantCulture = true,
        ErrorMessage = "El precio debe estar entre 0,01 y 9999,99.")]
    public decimal? Precio { get; set; }
}
```

El ejemplo muestra el **estado resultante después de activar las cuatro anotaciones**. No hace falta cambiar el controlador a la vez: la plataforma conoce cómo recoger los atributos del modelo de entrada al validar los valores recibidos. Tampoco tenemos que escribir JavaScript propio para expresar de nuevo los mismos límites cuando utilizamos los Tag Helpers y las bibliotecas de validación convencionales.

Los mensajes explícitos de las anotaciones tienen una ventaja didáctica: cuando falla una entrada, el alumno puede buscar el mismo texto en el modelo y en el HTML devuelto. La regla declarativa deja de ser un concepto abstracto y se convierte en una causa verificable del comportamiento de la aplicación.

Required no hace lo mismo que StringLength

Consideremos el nombre. Si no se envía el campo `Nombre`, `[Required]` comunica que falta un dato obligatorio. `[StringLength(60, MinimumLength = 2)]` no sustituye esa responsabilidad: su finalidad es imponer límites a la longitud cuando se dispone de una cadena. Son reglas complementarias.

Tampoco debemos confundir la longitud con el significado de la cadena. Un nombre de tres letras puede cumplir esas reglas y ser inapropiado para un negocio real; una cadena de cincuenta y nueve caracteres puede ser técnicamente válida sin constituir un nombre comercial aceptable. Las reglas de este curso son **criterios de demostración**. La definición final de las restricciones de un sistema de producción depende de sus requisitos y de su dominio.

En el reto opcional aparece un comportamiento interesante: `[Required]` aplicado a una cadena rechaza, de forma predeterminada, una entrada sin contenido útil, incluidas cadenas compuestas solo por espacios. Por eso no sería apropiado añadir una nueva condición en el controlador que repita sin explicación exactamente esa comprobación. Una regla adicional debe expresar algo que las anotaciones existentes no contemplan.

Required y Range sobre decimal?

El precio se representa con `decimal?`. Cuando no llega un valor, `[Required]` puede detectar su ausencia. Cuando el binder consigue convertir una cantidad numérica, `[Range]` evalúa si pertenece al intervalo permitido, entre 0,01 y 9999,99 en este ejemplo.

La sintaxis `[Range(typeof(decimal), "0.01", "9999.99", ParseLimitsInInvariantCulture = true)]` indica expresamente que **los límites textuales del atributo** se interpretan con la cultura invariante, evitando que `0.01` se convierta de forma distinta según la cultura del servidor. Es útil distinguir los límites definidos en el atributo de la forma en que el navegador representa las cantidades en distintas culturas. La conversión del dato recibido también depende de la cultura de la solicitud y del mecanismo de model binding. Un problema de separadores decimales no se arregla simplemente aumentando el máximo permitido.

Un precio igual a 0 es numérico, pero no cumple el intervalo. Un precio igual a -5 tampoco. Un texto como «abc», enviado a la propiedad Precio, tiene un problema de conversión antes incluso de comprobar si está entre los límites. Esto nos conduce a

una diferencia esencial: **los errores de model binding y los errores de validación son fuentes distintas de información** que confluyen en `ModelState`.

Model binding y model validation: un recorrido completo

Cuando llega un POST desde el formulario, MVC debe interpretar sus campos. Primero necesita saber cuál es la acción y cuál es su parámetro. En nuestro caso la firma es:

```
[HttpPost]
[ValidateAntiForgeryToken]
public IActionResult Crear(ProductoFormulario entrada)
```

El binder intenta construir `entrada` y asignar propiedades a partir de los valores enviados. Una entrada `Precio=7` puede convertirse en decimal; otra `Precio=abc` no es representable directamente en esa propiedad. Ese problema de conversión queda registrado como error de enlace.

Después interviene la validación del modelo, que examina atributos como `Required`, `StringLength` y `Range`. Una cadena de un solo carácter para Nombre sí puede convertirse en una cadena C#, pero no cumple nuestra longitud mínima. Por tanto, el problema no es de conversión, sino de **regla declarativa**.

En ASP.NET Core MVC, `ModelState` reúne información sobre ambos tipos de errores. Su propiedad `IsValid` permite conocer si la entrada está libre de los errores registrados. Esa comprobación es responsabilidad de la aplicación cuando construimos vistas MVC tradicionales; no debemos trasladar sin más lo que ocurre en una Web API que usa `[ApiController]`, donde el framework puede devolver automáticamente un 400 por validación.

La documentación de Microsoft explica que los errores de conversión y de reglas del modelo confluyen en ModelState y que, en MVC con vistas, la acción normalmente comprueba IsValid para devolver la página con mensajes de error. Validación de modelos en ASP.NET Core MVC.

Un error no implica automáticamente HTTP 400

Ésta es una distinción importante para el alumnado con experiencia en API REST. En nuestro formulario MVC, cuando `ModelState.IsValid` es falso, devolvemos la misma vista:

```
if (!ModelState.IsValid)
{
    return View(entrada);
}
```

En el escenario básico, el navegador suele recibir **200 OK** con el HTML de la página de creación y sus errores visibles. La respuesta no dice «el producto se ha creado»; indica que estamos mostrando el formulario de nuevo para que la persona pueda corregirlo. Un HTTP 200 por este camino no convierte los datos en válidos.

En cambio, una petición que incumple `[ValidateAntiForgeryToken]` normalmente obtiene **400 Bad Request** antes de que se ejecute el cuerpo de la acción. La diferencia no es accidental: una cosa es rechazar la legitimidad del envío y otra presentar un formulario normal con problemas de entrada.

Una tercera posibilidad es un 405 porque no hay una acción POST compatible con la ruta. Los tres síntomas —400, 405 y 200 con errores de validación— tienen causas diferentes y deben diagnosticarse consultando la petición, el código de la acción y el HTML devuelto.

La importancia de devolver la misma vista con sus datos

Cuando la entrada es incorrecta, sería frustrante obligar a la persona a volver a escribir todos los campos del formulario. Nuestro controlador utiliza `return View(entrada)` para que Razor vuelva a representar `Crear.cshtml` con el objeto recibido. La vista dispone de los Tag Helpers vinculados a `ProductoFormulario` y del contexto de `ModelState`.

El valor de un campo que era válido, como `Nombre="Carpeta grande"`, debe mantenerse al mostrar un error de precio. Incluso cuando hay un problema de conversión, los Tag Helpers de MVC pueden dar preferencia a los valores intentados almacenados en `ModelState`, permitiendo que el usuario vea el texto que escribió y pueda corregirlo. No hay que borrar todo el formulario por una propiedad inválida.

Conviene comprobar ese comportamiento con dos peticiones diferentes. Enviar `Nombre=Carpeta grande` y `Precio=-2` sirve para observar una regla de rango. Enviar `Nombre=Carpeta grande` y `Precio=abc` mediante una petición HTTP directa permite estudiar el error de conversión. El control `type="number"` del navegador puede impedir escribir «abc» en la interfaz normal, por eso es importante entender que **los clientes HTTP no se reducen a lo que permite escribir ese control**.

La vista utiliza `span asp-validation-for` para presentar los errores de cada propiedad:

```
<div>
    <label asp-for="Nombre"></label>
    <input asp-for="Nombre" />
    <span asp-validation-for="Nombre" class="text-danger"></span>
</div>
<div>
    <label asp-for="Precio"></label>
    <input asp-for="Precio" type="number" step="0.01" />
    <span asp-validation-for="Precio" class="text-danger"></span>
</div>
```

Si la validación de Nombre registra un error, su mensaje debe aparecer junto al campo correspondiente, no en un lugar arbitrario. El elemento span es parte del HTML que se produce; el atributo especial indica a Razor cuál es la propiedad asociada.

Validation Summary: ModelOnly no significa All

El formulario incluye también un elemento con `asp-validation-summary="ModelOnly"`. Hay que explicar su significado para evitar una interpretación errónea cuando no muestra los mismos mensajes que los `span`. **ModelOnly** se ocupa de errores asociados al modelo y no enumera de forma general todos los errores de propiedades. **All** permite mostrar una lista que incluya errores de propiedades y otros errores del modelo.

Por eso, cuando introducimos un precio fuera de rango, puede aparecer un mensaje junto a `Precio` sin que el resumen de `ModelOnly` lo repita. No significa que `ModelState` haya ignorado la regla. El alumnado deberá cambiar temporalmente `ModelOnly` a `All`, observar la diferencia y restaurar la primera opción para no duplicar mensajes sin necesidad.

Una validación legible no depende solamente de que exista un mensaje; también importa su relación con el campo y la posibilidad de corregirlo. Si un error se sitúa junto al precio y el valor escrito permanece visible, el usuario sabe qué debe revisar. El resumen general puede ser útil para errores que no correspondan a una única propiedad.

Conservar la separación entre la vista y las reglas

No sería una buena práctica escribir la regla de precio dos veces: una en el controlador y otra, con valores distintos, en JavaScript. Al declarar las reglas de

entrada en `ProductoFormulario`, los Tag Helpers pueden generar metadatos de validación en atributos HTML, y las bibliotecas cliente compatibles pueden interpretarlos.

Por ejemplo, al usar un modelo con DataAnnotations, el HTML de un input puede contener atributos `data-val` que describen restricciones. Estos atributos no son por sí mismos un algoritmo de validación ejecutable: tienen que ser interpretados por un script compatible para ofrecer validación anticipada en el navegador. ASP.NET Core conserva la validación de servidor independientemente de ese componente opcional.

Nuestro proyecto no añade un sistema moderno de frontend, ni un gestor complejo de formularios, ni componentes Blazor en esta jornada. En cambio, introduce una demostración limitada con jQuery, jQuery Validation y jQuery Validation Unobtrusive. Es una combinación convencional para observar la **validación discreta** de formularios MVC.

El contrato esencial sigue siendo el mismo. Una petición inválida debe ser rechazada desde la acción mediante la comprobación de `ModelState.IsValid` aunque se deshabilite JavaScript o no se cargue ningún recurso externo. La validación en cliente es una ayuda para presentar errores antes del viaje HTTP, **no una garantía de seguridad**.

Validación en el navegador y su dependencia de los scripts

La versión de proyecto de P2.3 incorpora un archivo `Views/Shared/_ValidationScriptsPartial.cshtml` con referencias a tres recursos JavaScript: jQuery, jQuery Validation y jQuery Validation Unobtrusive. Las versiones concretas están fijadas en el archivo para facilitar la reproducción del laboratorio. Las bibliotecas deben cargarse en un orden coherente, ya que unas dependen de las otras.

En el layout encontramos:

```
@await RenderSectionAsync("Scripts", required: false)
```

Esa instrucción permite que una vista declare su propia sección opcional de scripts. El atributo `required: false` evita que otras vistas del catálogo deban declarar esa sección. No se altera la acción del controlador ni el modelo por añadir un punto de inserción de recursos JavaScript en el layout.

La vista `Crear.cshtml` contiene, inicialmente comentada, esta sección que el alumno puede activar después de haber comprobado la validación de servidor:

```
@section Scripts
{
    <partial name="_ValidationScriptsPartial" />
}
```

Cuando se habilita, se representa la parcial correspondiente y el navegador intenta descargar los scripts. En un laboratorio con acceso a la CDN indicada, las bibliotecas interpretan los atributos generados a partir de DataAnnotations y pueden mostrar mensajes antes de enviar una solicitud inválida. Es frecuente que un formulario con un valor incorrecto no llegue a originar el POST cuando la validación cliente lo intercepta.

Esto no convierte el laboratorio en una prueba de seguridad. **Debe repetirse la misma entrada inválida con JavaScript deshabilitado o mediante una petición HTTP directa** para comprobar que el servidor sigue rechazándola. Si el navegador no tiene acceso a la CDN, la demostración de cliente no podrá completarse hasta proporcionar archivos locales apropiados; en cambio, los controles de servidor deberían continuar funcionando. El resultado dependiente de una CDN no debe presentarse como si estuviese garantizado en entornos sin conexión.

Las bibliotecas y el patrón de validación cliente se describen también en la documentación oficial de Microsoft: https://learn.microsoft.com/en-us/aspnet/core/mvc/models/validation?view=aspnetcore-8.0.

Por qué el navegador no sustituye al servidor

Es posible que el control `type="number"` no permita introducir letras en una interfaz concreta. Otro navegador puede representar los controles de forma distinta y un cliente HTTP puede enviar cualquier cadena como valor de campo. Las reglas críticas tienen que seguir verificándose en el servidor. Un mensaje anticipado del navegador mejora la experiencia de usuario, pero no constituye una frontera confiable.

Por la misma razón, no añadiremos al controlador un camino de inserción que omita `ModelState` simplemente porque la vista ya valida antes de enviar. Al llegar P2.4, donde el POST pasará a escribir en una base real, esta disciplina se volverá aún más importante: **no se guardarán productos procedentes de entradas inválidas**.

Un ejemplo completo: nombre válido y precio negativo

Volvamos al formulario de producto. Un usuario envía los campos `Nombre=Rotulador` y `Precio=-5` utilizando un POST protegido. Supongamos que las DataAnnotations ya están activadas. El binder obtiene una cadena y un decimal; la conversión ha sido posible. El problema aparece después: el decimal no cumple el mínimo 0,01 que establece `[Range]`.

La acción consulta `ModelState.IsValid` y obtiene `false`. Entonces devuelve `View(entrada)`. Razor genera otra vez `Crear.cshtml`, conserva el nombre escrito y añade junto a Precio el mensaje «El precio debe estar entre 0,01 y 9999,99». No se representa Confirmacion.cshtml y no se crea ninguna entidad de dominio.

Si el usuario corrige el precio a 7 y vuelve a enviar, los valores ya cumplen las reglas del formulario. La acción puede construir `Producto` y devolver la confirmación. Como P2.3 no contiene operaciones de almacenamiento, el catálogo continúa mostrando los tres elementos originales.

En ese pequeño escenario aparecen todos los conceptos de los tres primeros puntos del M02. P2.1 enseñó a recibir los valores. P2.2 los relacionó de forma declarativa con la vista. P2.3 decide si son aceptables y permite corregirlos. Esta acumulación progresiva justifica trabajar sobre una sola aplicación en vez de presentar tres demostraciones inconexas.

Variaciones que revelan causas distintas

Nombre vacío y precio 7. La anotación Required detecta que falta el nombre. La vista debe mostrar el mensaje asociado a Nombre, sin borrar de forma innecesaria el valor de precio que sí era correcto.

Nombre «X» y precio 7. Existe un nombre, pero no satisface la longitud mínima. El mensaje procede de StringLength, no de Required.

Nombre «Rotulador» y precio 0. Existe un valor numérico, pero está fuera del rango permitido. Required no basta para resolver esa situación.

Nombre «Rotulador» y precio «abc». El binder no puede representar el texto como decimal; ModelState registra un problema de conversión. El navegador normal puede impedir introducir esas letras en el control, pero un cliente HTTP puede remitirlas. Esta variante pertenece al diagnóstico de entradas, no a una regla de negocio Range.

Formulario sin token antiforgery. La petición puede ser rechazada con 400 antes de que el cuerpo de la acción valide esos campos. No debemos interpretar ese 400 como la respuesta que representa un error ordinario de Name o Precio dentro de la misma vista.

En el material práctico cada situación se trabaja con una predicción previa, una modificación concreta cuando corresponda y una comprobación visible. El objetivo es que el estudiante pueda decir **qué capa actuó y por qué**, no que memorice una tabla de códigos HTTP sin comprender el recorrido.

Cómo diagnosticar ModelState sin convertir el log en la solución

Cuando un formulario muestra un error, el alumno dispone de varios puntos de observación: las herramientas de red del navegador permiten ver lo que se envió; el código del modelo revela las anotaciones; la acción indica cómo se comprueba IsValid; y la vista explica dónde se representan mensajes. A veces es útil inspeccionar también las claves y errores registrados en ModelState.

El proyecto contiene un bloque opcional comentado que recorre las entradas de `ModelState` que tienen errores y escribe sus mensajes mediante `System.Diagnostics.Debug.WriteLine`. El alumno puede activarlo de forma temporal, ejecutar una solicitud inválida y comprobar que las claves `Nombre` y `Precio` corresponden a las propiedades declaradas. El ejercicio no exige introducir un sistema de logging adicional ni modificar la configuración del laboratorio.

Esta inspección debe diferenciarse de la solución de negocio. Observar `ModelState` no implica aceptar un valor inválido; la decisión del controlador sigue dependiendo de IsValid. Tampoco resulta útil imprimir en un log de producción todas las entradas del formulario si pueden contener datos personales: en esta demostración se registran mensajes de error y nombres de campos, no un volcado indiscriminado de la petición.

Aplicación de transferencia: Display y el texto de una etiqueta

Además de reglas de validación, las DataAnnotations permiten aportar información de presentación, por ejemplo mediante `[Display(Name = "Nombre del producto")]`. Este atributo no necesita renombrar la propiedad de C#. El modelo puede conservar `Nombre` y el Tag Helper `asp-for="Nombre"` puede generar un texto de etiqueta más descriptivo para el usuario.

La aplicación autónoma de P2.3 consistirá en escribir ese atributo sobre la propiedad correspondiente y observar el resultado. El alumno deberá explicar por qué el input continúa enviando el campo `Nombre`, mientras que la etiqueta visual puede mostrar «Nombre del producto». Es una comprobación de que **los metadatos de presentación y el contrato de transporte HTTP no son lo mismo**.

No convertiremos la actividad en una investigación sobre todos los atributos de DataAnnotations. Nos basta con una observación funcional conectada a los Tag Helpers que ya conocemos y al nuevo modelo de entrada.

Reto opcional: un error contextual añadido al ModelState

Las anotaciones sencillas son apropiadas para restricciones que pueden expresarse junto a una propiedad. Sin embargo, existen situaciones en que la acción necesita señalar un problema contextual que no viene de una anotación. Como demostración controlada, el reto opcional marcará como inválidos los nombres que comiencen por `TEST`.

El controlador contiene un bloque comentado que utiliza `ModelState.AddModelError(nameof(entrada.Nombre), ...)` cuando se cumple esa condición. El alumno podrá activarlo, introducir un nombre que tenga longitud válida y observar que, aun cumpliendo Required y StringLength, el formulario muestra un nuevo error de Nombre y no pasa a la confirmación.

El ejemplo sirve para conocer la mecánica, **no para sugerir que todas las reglas empresariales deban escribirse en controladores**. En aplicaciones complejas conviene separar la lógica del dominio, los permisos y las validaciones entre distintos componentes. Nuestro curso reducido no necesita desarrollar toda esa arquitectura: únicamente debe mostrar con claridad que `ModelState` puede recoger diferentes tipos de errores y que la vista puede representarlos de forma consistente.

Una extensión opcional consistiría en estudiar si una regla personalizada merece vivir en un atributo de validación, en un validador específico o en un servicio de dominio. No implementaremos esas tres alternativas aquí, porque desplazarían las tareas esenciales del CRUD del módulo.

Conexión de P2.3 con el CRUD de P2.4

Al terminar este capítulo, el catálogo sigue funcionando sobre datos de ejemplo, pero la recepción de productos ya dispone de un contrato tipado y reglas de aceptación. Podemos distinguir los errores que corresponden al enlace de modelos de los que corresponden a DataAnnotations, devolver la misma vista con mensajes, conservar la entrada escrita y comprobar la validación tanto en el servidor como, cuando se habilitan y cargan los scripts, en el navegador.

El siguiente cambio será de naturaleza diferente. En **2.4** incorporaremos SQL Server Express LocalDB y EF Core para crear, listar, editar y eliminar productos. Las propiedades de entrada seguirán perteneciendo a `ProductoFormulario`, mientras que las operaciones persistentes actuarán sobre la entidad `Producto`. La regla que hemos aprendido —**no convertir una entrada inválida en un cambio de estado**— se aplicará entonces a operaciones que sí afectan a la base de datos.

No se introducirán migraciones avanzadas, arquitecturas adicionales ni repaso extenso de EF Core, porque ese conocimiento es un requisito previo del curso. La atención se concentrará en la integración del formulario Razor con un CRUD MVC pequeño, así como en la diferencia entre presentar un resultado temporal y persistir un objeto que seguirá disponible después de reiniciar la aplicación.

Documentación oficial relacionada

- Microsoft Learn, Validación de modelos en MVC y Razor Pages: https://learn.microsoft.com/en-us/aspnet/core/mvc/models/validation?view=aspnetcore-8.0
- Microsoft Learn, Tutorial MVC con validación de entradas: https://learn.microsoft.com/en-us/aspnet/core/tutorials/first-mvc-app/validation?view=aspnetcore-8.0
- Microsoft Learn, Model binding en MVC: https://learn.microsoft.com/en-us/aspnet/core/mvc/models/model-binding?view=aspnetcore-8.0
- Microsoft Learn, Formularios y Tag Helpers: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/working-with-forms?view=aspnetcore-8.0

Profundización aplicada: dos fases de validación y una sola decisión de negocio

La palabra «validación» designa operaciones relacionadas, pero no idénticas. Primero el framework intenta **construir el dato**: debe interpretar las cadenas HTTP

y asignarlas a propiedades con tipos C#. Después comprueba **restricciones declaradas** sobre ese dato: longitud del nombre, obligatoriedad o intervalo permitido para el precio. Una conversión fallida puede producir errores de `ModelState` aunque todavía no se haya ejecutado una regla `[Range]` sobre un `decimal` válido. Esa diferencia permite entender por qué algunos mensajes proceden del binder y otros de las DataAnnotations.

Un diseño habitual es utilizar un modelo de entrada separado de la entidad de persistencia. El formulario no debería decidir el identificador autogenerado, la fecha de creación o cualquier propiedad de dominio que el servidor controla por sí mismo. Con `ProductoFormulario`, el contrato HTTP declara únicamente lo que la persona puede aportar: nombre y precio. Esta separación reduce el riesgo de sobrepublicación, pero exige un paso explícito en el controlador que copie los valores aceptados a la entidad. En 2.4 veremos esa traducción antes de escribir en LocalDB.

El tipo anulable no significa ausencia de validación. Cuando declaramos `decimal? Precio`, expresamos que el binder puede no construir todavía un valor. `[Required]` establece si ese estado es aceptable para el formulario. `[Range]` expresa el intervalo válido una vez que hay un número interpretable. Ambas responsabilidades deben aparecer separadas en nuestro razonamiento. Si utilizáramos una propiedad no anulable `decimal`, su valor predeterminado puede ocultar la distinción entre campo no enviado y cero introducido; el diseño del DTO del formulario ayuda a comunicar esa intención.

Las culturas numéricas añaden un problema distinto. «4,50» y «4.50» pueden interpretarse de manera diferente según la cultura que participa en el binding y las convenciones de la interfaz. La anotación `Range` con límites decimales debe construirse de manera coherente, y la implementación del proyecto utiliza interpretación invariante para sus límites. Esa decisión se refiere a los **límites de la regla**, no promete que toda combinación de navegador, cultura y valor escrito vaya a convertirse de forma idéntica. Para un laboratorio reproducible, un precio entero como 7 elimina una variable que no necesitamos en cada prueba.

Una política de error es más útil cuando describe la corrección posible. «El nombre debe tener entre 2 y 60 caracteres» identifica el campo y sus límites; «Introduce un nombre» diferencia ausencia de contenido respecto de longitud. Si la validación falla, devolver la misma vista con el modelo de entrada permite conservar las correcciones ya realizadas. En nuestro controlador, el orden es esencial: comprobar `ModelState.IsValid` **antes** de convertir el DTO a entidad y de ejecutar persistencia. De

lo contrario, se podría guardar un dato inválido aunque la interfaz terminara mostrando errores.

La validación de navegador mejora la experiencia, pero funciona en un entorno controlado por la persona que envía la petición. Se puede desactivar JavaScript, invocar HTTP con otra herramienta o manipular el cuerpo enviado. Por ello, una aplicación que únicamente «impide pulsar Enviar» no está validando de forma suficiente su operación de servidor. Las reglas de negocio críticas deben ejecutarse en el servidor. Esta idea enlaza con antiforgery: un token válido demuestra que la petición supera una comprobación de procedencia, **no** que sus valores sean correctos. Son filtros diferentes y pueden fallar de manera independiente.

Caso de diagnóstico razonado. Supongamos que un usuario envía `Precio=abc` y obtiene una vista con un mensaje de conversión, mientras otro envía `Precio=0` y obtiene el texto de `Range`. El primer síntoma apunta a la construcción de un `decimal?`; el segundo a una regla sobre un valor ya construido. Si un tercero envía `Nombre=X` con un precio correcto, el error será de longitud. Ninguna de estas observaciones exige todavía una consulta SQL. La separación entre recepción, conversión, validación y persistencia es precisamente la estructura mental que permite resolver las incidencias sin confundir capas.

2.4. CRUD persistente de productos con EF Core y SQL Server Express LocalDB

Cuando una confirmación deja de ser suficiente

En P2.1 aprendimos a recibir un producto mediante un POST protegido. En P2.2 mejoramos la generación del formulario con Tag Helpers; en P2.3 diferenciamos un dato recibido de un dato válido, mediante un modelo de entrada, DataAnnotations y `ModelState`. Hemos progresado sin cambiar de aplicación ni introducir dependencias que ocultaran la responsabilidad de cada paso. Sin embargo, el usuario todavía podía preguntar: **«He enviado un producto correcto; ¿por qué no aparece cuando vuelvo al catálogo?»**. Hasta ahora, la respuesta era que el controlador solamente devolvía una vista de confirmación: el estado persistente no se modificaba.

El problema de 2.4 consiste en conectar la interfaz MVC que ya conocemos con una fuente de datos que sobreviva a las peticiones HTTP y al reinicio del proceso. Utilizaremos **Entity Framework Core 8 y SQL Server Express LocalDB**. El alumnado ya conoce el funcionamiento general de EF Core y C#, de modo que no necesitamos repetir un curso de mapeo objeto-relacional: nos concentraremos en las decisiones específicas de una aplicación MVC con formularios, validación, vistas Razor, identificadores de ruta y respuestas HTTP.

El resultado del itinerario esencial será un **CRUD**: consultar una lista y un detalle (**Read**), crear (**Create**), editar (**Update**) y eliminar (**Delete**) productos. No se desarrollarán roles empresariales, autenticación Identity, servicios distribuidos, mecanismos de concurrencia avanzados ni un repositorio de acceso a datos artificial. Esas cuestiones tienen valor propio, pero exceden el alcance de seis horas del segundo día y pueden distraernos de la conexión entre un formulario Razor y una entidad persistente.

Antes de tocar SQL Server debemos definir qué significa aquí **persistir**. Si enviamos Nombre = «Rotulador» y Precio = 7, el nuevo producto debe aparecer en `/Productos`, disponer de un identificador que permita abrir `/Productos/Detalle/{id}` y seguir existiendo después de detener y arrancar de nuevo el servidor. El mismo principio se aplica a una modificación de precio o a una eliminación. Una página de confirmación sin guardar no cumple ese requisito, por muy correcto que sea su HTML.

Una sola aplicación y una base LocalDB deliberadamente sencilla

El proyecto conserva la estructura básica de MVC: controladores en `Controllers/`, tipos de entrada y dominio en `Models/`, vistas en `Views/`, configuración de servicios y middleware en `Program.cs`. Añadimos una carpeta `Data/` con el contexto EF Core y configuramos una conexión SQL Server. No estamos creando una API auxiliar ni trasladando la lógica al navegador.

SQL Server Express LocalDB es una tecnología ligada al entorno Windows. En nuestro laboratorio permite disponer de una instancia local sin explicar la administración de un servidor SQL remoto. No debe interpretarse como una solución directamente transportable a contenedores Linux: en M05, cuando comparemos Kestrel, IIS y Docker, utilizaremos demostraciones compatibles con el entorno concreto y explicaremos esta limitación.

La cadena de conexión se encuentra bajo el nombre `Productos` en `appsettings.json`:

```
{
  "ConnectionStrings": {
    "Productos": "Server=(localdb)\
\MSSQLLocalDB;Database=CursoWebProductos;Trusted_Connection=True;TrustServerCertifica
te=True;MultipleActiveResultSets=true"
  }
}
```

En este archivo JSON, la barra inversa del nombre de instancia aparece escapada como `\\`. En la cadena SQL resultante designa la instancia `(localdb)\\MSSQLLocalDB`. La base se llama `CursoWebProductos`. El atributo `Trusted_Connection=True` permite usar autenticación integrada, apropiada para un ejercicio local ejecutado con la cuenta de Windows del alumno; no estamos preparando credenciales de despliegue ni almacenando una contraseña en el repositorio.

Importante para el laboratorio: cada persona debe utilizar su propia instancia y base local. Si ya ha usado una base con ese nombre en ejercicios anteriores, su contenido y esquema podrían diferir del estado inicial descrito aquí. Cambiar el nombre de la base a uno nuevo para la práctica evita interferencias sin borrar datos de otros ejercicios. No se debe recomendar destruir bases de datos desconocidas «para empezar de cero». Más adelante mostraremos cómo se puede suministrar una cadena alternativa con la variable `ConnectionStrings__Productos`, sin editar el fichero distribuido.

Para comprobar la instalación en Windows puede utilizarse `sqllocaldb info` desde una terminal. Que el comando exista no demuestra que la aplicación pueda conectarse: también influyen la instancia, los permisos y la configuración. La observación decisiva será el arranque de la aplicación y la consulta real del catálogo.

Producto y ProductoFormulario: conservar lo aprendido al escribir en la base

La entidad del catálogo conserva tres propiedades:

```
public sealed class Producto
{
    public int Id { get; set; }
    public string Nombre { get; set; } = string.Empty;
    public decimal Precio { get; set; }
}
```

Esta clase representa el dato persistente: `Id` identifica una fila, `Nombre` describe el producto y `Precio` representa una cantidad. EF Core podrá interpretar `Id` como clave primaria mediante sus convenciones habituales. En SQL Server, ese identificador podrá generarse al insertar el registro. La acción de alta **no necesita recibir un Id elegido por el navegador**.

En cambio, el formulario seguirá usando el contrato validado de P2.3:

```
public sealed class ProductoFormulario
{
    [Required(ErrorMessage = "Indica el nombre.")]
    [StringLength(60, MinimumLength = 2,
        ErrorMessage = "El nombre debe tener entre 2 y 60 caracteres.")]
    public string? Nombre { get; set; }

    [Required(ErrorMessage = "Indica el precio.")]
    [Range(typeof(decimal), "0.01", "9999.99",
        ParseLimitsInInvariantCulture = true,
        ErrorMessage = "El precio debe estar entre 0,01 y 9999,99.")]
    public decimal? Precio { get; set; }
}
```

La propiedad `Nombre` se mantiene anulable en el tipo de entrada para representar el caso en el que falta el campo, y `Precio` es `decimal?` para distinguir un número de una ausencia. Las anotaciones declaran las restricciones: nombre obligatorio de dos a sesenta caracteres y precio dentro de un intervalo ilustrativo. El argumento `ParseLimitsInInvariantCulture = true` se refiere a la interpretación de los límites textuales

del atributo; **no configura por sí solo la cultura de todas las solicitudes HTTP**.

El valor de esta separación no es «tener más clases». Su valor es definir **qué acepta el servidor desde un formulario**. Un usuario no debe poder elegir arbitrariamente la identidad del producto nuevo ni modificar otras propiedades futuras solo añadiendo campos ocultos a la petición. En un proyecto más grande esta decisión puede formar parte de un sistema de DTOs, pero aquí una clase concreta es suficiente. Evita introducir una capa de mapeo adicional para copiar únicamente dos propiedades.

La ruta de edición sigue la misma regla: tomamos el `id` de la ruta, buscamos la entidad que existe y asignamos explícitamente las propiedades aprobadas del `ProductoFormulario`. No recibimos un `Producto` completo para marcarlo como modificado sin revisar su procedencia.

DbContext: una dependencia de los controladores, no de las vistas

La nueva clase `AppDbContext`, en `Data/AppDbContext.cs`, hereda de `DbContext`. Expone el conjunto `Productos`:

```
public sealed class AppDbContext(DbContextOptions<AppDbContext> options)
    : DbContext(options)
{
    public DbSet<Producto> Productos => Set<Producto>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Producto>()
            .Property(p => p.Precio).HasPrecision(10, 2);
    }
}
```

No necesitamos explicar todos los estados internos del change tracker para comprender qué representa este contexto: es la pieza que utiliza EF Core para construir consultas y administrar cambios sobre nuestra base. La configuración `HasPrecision(10, 2)` especifica una precisión coherente con el uso de precios de dos decimales. No significa que la aplicación haya implementado cálculos financieros rigurosos ni sustituye la validación del intervalo en el formulario.

El controlador declara la dependencia mediante su constructor:

```
private readonly AppDbContext _db;

public ProductosController(AppDbContext db) => _db = db;
```

ASP.NET Core puede resolver esta dependencia porque registramos el contexto en el contenedor de servicios:

```
var cadena = builder.Configuration.GetConnectionString("Productos")
    ?? throw new InvalidOperationException("Falta ConnectionStrings:Productos.");

builder.Services.AddDbContext<AppDbContext>(
    options => options.UseSqlServer(cadena));
```

Este es un **punto de continuidad con M01**: allí aprendimos que el host registra servicios antes de construir la aplicación. Ahora registramos uno más. La acción no abre manualmente la conexión con `new SqlConnection`, y la vista no crea un `DbContext`. El contenedor proporciona el contexto a la aplicación con el ámbito habitual de la solicitud, y el controlador trabaja con esa dependencia. Documentación EF Core sobre DbContext y DI.

La pregunta pedagógica relevante es: «¿qué ganamos al recibir `AppDbContext` en lugar de instanciarlo directamente?». La configuración permanece centralizada y las acciones no necesitan conocer cómo se construye una conexión. Cuando el alumno abra `ProductosController`, encontrará operaciones sobre objetos de dominio y un contexto ya suministrado, no cadenas SQL repetidas en cada método.

Crear el esquema sin introducir un segundo curso de migraciones

El proyecto contiene un bloque de inicialización de laboratorio:

```
using (var scope = app.Services.CreateScope())
{
    var db = scope.ServiceProvider.GetRequiredService<AppDbContext>();
    bool baseCreada = db.Database.EnsureCreated();

    if (baseCreada)
    {
        db.Productos.AddRange(
            new Producto { Nombre = "Cuaderno", Precio = 4.50m },
            new Producto { Nombre = "Lápiz", Precio = 1.20m },
            new Producto { Nombre = "Carpeta", Precio = 3.75m });
        db.SaveChanges();
```

```
    }
}
```

El contexto se obtiene dentro de un ámbito explícito porque esta inicialización tiene lugar durante el arranque del host, fuera de una solicitud HTTP. `EnsureCreated` permite **crear una base nueva con su esquema** si todavía no existe. El valor booleano indica si la base se creó en esa operación; solamente en ese caso se insertan los tres registros de demostración. Esa condición no es ornamental: si el alumno elimina productos y reinicia Kestrel, el programa **no debe recrear los datos eliminados**.

Es importante explicar los límites de `EnsureCreated`. No lleva un historial de migraciones ni actualiza automáticamente un esquema anterior cuando cambia el modelo. **No se debe mezclar `EnsureCreated` con el flujo normal de migraciones `Migrate` sobre la misma base**. Para aplicaciones que evolucionan y se despliegan de forma sostenida, las migraciones tienen otro propósito. Aquí escogemos un estado inicial simplificado porque el alumnado ya sabe EF Core y el curso necesita destinar tiempo a los formularios MVC. Documentación EF Core sobre EnsureCreated.

La inicialización no es una operación apropiada para ejecutar en cada petición ni para copiar sin revisión a una aplicación de producción. Si LocalDB no está disponible, el programa puede fallar antes de empezar a escuchar en Kestrel porque la inicialización necesita conectarse. En ese caso, no debemos interpretar el error como un fallo de routing o una vista que no existe; la causa se encuentra en la dependencia SQL.

El primer arranque es una observación, no solo un trámite

Cuando el estudiante ejecuta la aplicación por primera vez sobre una base nueva, debería observar que la lista muestra Cuaderno, Lápiz y Carpeta. Después puede detener Kestrel, volver a iniciarlo y comprobar que esos mismos datos no se duplican. La semilla únicamente se aplica al crear la base. Esta comprobación anticipa el concepto de persistencia incluso antes de activar Create.

Si la lista aparece vacía al usar una base que ya existía, no es correcto deducir automáticamente que EF Core haya borrado los datos. Debemos inspeccionar qué base se está utilizando, si su esquema corresponde al proyecto y si había filas previas. Tampoco debemos borrar sin confirmar una base con el mismo nombre perteneciente a otro trabajo. El laboratorio tiene que ser reproducible **sin poner en riesgo otros datos del alumno**.

Read: reutilizar Index y Detalle sobre consultas reales

En M01, la acción `Index` creaba o recuperaba una lista fija de tres productos. En P2.4 la acción consulta el conjunto EF Core:

```
[HttpGet]
public async Task<IActionResult> Index()
{
    var productos = await _db.Productos
        .AsNoTracking()
        .OrderBy(p => p.Id)
        .ToListAsync();

    return View(productos);
}
```

El método sigue devolviendo una vista Razor con una colección. Esto nos permite conservar los conocimientos de presentación, parciales, listas y enlaces adquiridos durante M01. Lo que cambia es **la procedencia de los datos**, no el motor que genera el HTML. El método de consulta es asíncrono, porque el acceso al proveedor de datos implica trabajo de entrada y salida; `ToListAsync` materializa el resultado de la consulta.

`AsNoTracking()` indica que no necesitamos que el contexto haga seguimiento de las entidades que estamos recuperando solamente para mostrarlas. Se trata de una decisión adecuada para listas y detalles de lectura. En cambio, en las operaciones de edición y eliminación recuperaremos una entidad que el contexto pueda seguir para aplicar `SaveChangesAsync`.

`OrderBy(p => p.Id)` da a los resultados un orden explícito y reproducible. No debemos asumir que SQL Server devolverá siempre los registros en orden de inserción si no solicitamos una ordenación. En nuestros primeros ejercicios es útil que Cuaderno, Lápiz y Carpeta aparezcan de forma estable, pero esa elección se puede cambiar más adelante.

La acción de detalle recibe un identificador desde el routing y devuelve `NotFound()` cuando no existe un producto con ese valor:

```
[HttpGet]
public async Task<IActionResult> Detalle(int id)
{
    if (id <= 0)
        return BadRequest("El identificador debe ser mayor que cero.");
```

```
    var producto = await _db.Productos.AsNoTracking()
        .FirstOrDefaultAsync(p => p.Id == id);

    return producto is null ? NotFound() : View(producto);
}
```

La distinción entre **400 por un identificador estructuralmente inválido** y **404 por un identificador positivo que no existe** resulta más clara ahora que la fuente de datos puede crecer, cambiar o perder registros. El enlace generado en el catálogo incorpora el Id de cada entidad. La vista no busca en SQL Server; el controlador decide qué objeto le entrega.

Create: transformar un POST válido en un INSERT persistente

En P2.3 una solicitud correcta terminaba en `View("Confirmacion", producto)`. Esa vista era útil para estudiar el binder, pero la entidad se perdía como nuevo registro porque no la habíamos guardado. En P2.4 activaremos una acción POST que crea una entidad y llama a `SaveChangesAsync`.

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Crear(ProductoFormulario entrada)
{
    if (!ModelState.IsValid)
        return View(entrada);

    var producto = new Producto
    {
        Nombre = entrada.Nombre!,
        Precio = entrada.Precio!.Value
    };

    _db.Productos.Add(producto);
    await _db.SaveChangesAsync();

    return RedirectToAction(nameof(Detalle), new { id = producto.Id });
}
```

El fragmento reproduce el bloque que el alumno activará en `ProductosController`. Observa que **no desaparece la protección antiforgery ni la comprobación de ModelState**: la persistencia se añade *después* de haber aprendido a recibir y validar. La aplicación no debería escribir un producto inválido simplemente porque existe una base disponible.

El método `Add` informa al contexto de que queremos crear una entidad nueva. La llamada `SaveChangesAsync` materializa el cambio en la base de datos. Antes de guardar, `producto.Id` representa su valor temporal o predeterminado; después de guardar, SQL Server habrá asignado el identificador persistente mediante el mecanismo configurado para la clave. No hay que inventar el número 4 como hicimos en una confirmación temporal de P2.3.

La última línea merece atención especial: **no devuelve directamente otra vista**. Utiliza `RedirectToAction` para indicar al navegador que realice una petición GET a la ficha del producto creado. Este patrón es conocido como **Post/Redirect/Get (PRG)**. Si el usuario actualiza la página final, se recarga un GET de detalle en lugar de repetir automáticamente un POST de inserción. No constituye una garantía universal contra duplicados o envíos concurrentes, pero organiza de forma adecuada el flujo habitual de un formulario.

La secuencia observable será POST protegido a `/Productos/Crear`, **302** de redirección y GET a `/Productos/Detalle/{id}`. Si el navegador sigue automáticamente la redirección, normalmente el usuario verá el HTML de detalle sin prestar atención al 302 intermedio. Por eso, para estudiar el protocolo, conviene utilizar la pestaña Red y observar las dos solicitudes por separado.

Qué pasa cuando la validación falla

Si el nombre tiene un solo carácter o el precio es cero, `ModelState.IsValid` es falso y se ejecuta `return View(entrada)`. El controlador **no llega** a `_db.Productos.Add` ni a `SaveChangesAsync`. La vista muestra mensajes de error y conserva los valores admitidos. El alumno debe comprobar que el catálogo no contiene un producto nuevo y que el número de filas no aumenta tras una petición inválida.

La validación servidor-cliente sigue funcionando como en P2.3. No vamos a «optimizar» la acción eliminando el `if` para confiar únicamente en los atributos HTML. Un POST puede proceder de un cliente sin JavaScript y debe someterse a las mismas reglas del modelo de entrada.

El problema de una segunda solicitud idéntica

PRG facilita la navegación tras un envío correcto, pero no impide que un usuario realice deliberadamente dos POST válidos desde dos pestañas o envíe dos veces la misma información. La base puede contener registros duplicados si no existen reglas de unicidad. En este ejercicio **no inventaremos un índice único ni un**

algoritmo de idempotencia que no figura en el temario; el objetivo es comprender el flujo persistente de MVC.

Una ampliación opcional de otro curso podría definir qué significa «producto duplicado» y qué política de negocio debe aplicarse. Aquí basta con reconocer que un patrón de navegación y una restricción de datos resuelven problemas distintos.

Update: cargar primero, editar después y guardar un cambio concreto

La edición no es una creación con un botón diferente. Para editar correctamente hay que identificar una entidad **que ya existe**. El formulario GET de `Editar(int id)` busca el producto y presenta sus valores en un `ProductoFormulario`. El POST vuelve a recibir ese formulario validado, busca la entidad y modifica solo las propiedades permitidas.

```
[HttpGet]
public async Task<IActionResult> Editar(int id)
{
    if (id <= 0) return BadRequest();

    var producto = await _db.Productos.AsNoTracking()
        .FirstOrDefaultAsync(p => p.Id == id);
    if (producto is null) return NotFound();

    ViewData["ProductoId"] = producto.Id;
    return View(new ProductoFormulario
    {
        Nombre = producto.Nombre,
        Precio = producto.Precio
    });
}
```

El GET utiliza `AsNoTracking()` porque solo necesita poblar el formulario inicial. Guarda el identificador en `ViewData["ProductoId"]` para construir en la vista la URL del POST. Hay alternativas más estructuradas, como un modelo específico que incluya el Id de navegación, pero este ejemplo minimiza clases adicionales para centrarse en la diferencia entre **identificar un recurso y editar sus campos**.

En la vista `Editar.cshtml`, el Form Tag Helper declara `asp-action="Editar"`, `asp-route-id="@productoId"` y `method="post"`. El Id se suministra como valor de ruta, mientras que `Nombre` y `Precio` proceden de los inputs. El controlador no acepta que el navegador cambie cualquier propiedad de la entidad de dominio mediante un formulario.

El POST ejecuta otra consulta, esta vez sin `AsNoTracking()`:

```
[HttpPost]
[ValidateAntiForgeryToken]
public async Task<IActionResult> Editar(int id, ProductoFormulario entrada)
{
    if (id <= 0) return BadRequest();

    var producto = await _db.Productos
        .FirstOrDefaultAsync(p => p.Id == id);
    if (producto is null) return NotFound();

    if (!ModelState.IsValid)
    {
        ViewData["ProductoId"] = id;
        return View(entrada);
    }

    producto.Nombre = entrada.Nombre!;
    producto.Precio = entrada.Precio!.Value;

    await _db.SaveChangesAsync();
    return RedirectToAction(nameof(Detalle), new { id });
}
```

Ahora `producto` es una entidad seguida por el contexto. Al asignar las dos propiedades y guardar, EF Core puede emitir la operación de actualización. No necesitamos llamar a `Update` sobre un objeto recibido arbitrariamente desde la solicitud: **leemos la entidad existente y actualizamos explícitamente únicamente los campos permitidos**. Este patrón reduce la superficie de sobrepublicación y resulta pedagógicamente transparente.

Por qué restauramos el Id del formulario cuando hay errores

Cuando el POST de edición recibe un precio inválido, devuelve la misma vista con `return View(entrada)`. Pero `Editar.cshtml` necesita conocer **a qué producto debe volver a enviar** el formulario cuando la persona corrija el dato. Por eso el controlador restaura `ViewData["ProductoId"] = id` antes de devolver la vista inválida.

Si olvidamos ese paso, el formulario podría generar una URL sin el identificador correcto, aunque los mensajes de validación aparezcan. La persona intentaría corregir una propiedad, pero el segundo POST podría dirigirse a un destino distinto. El error demuestra que la trazabilidad de una operación de edición incluye tanto el modelo de entrada como la **identidad del recurso**.

La edición también debe devolver 404 si alguien solicita un Id positivo que ya no existe, y no debe crear un producto nuevo con ese número. La creación y la modificación son operaciones distintas. Este ejemplo no resuelve conflictos de concurrencia entre usuarios que cambien simultáneamente la misma fila; ese escenario se podría abordar en una formación específica de EF Core.

Delete: una pantalla de confirmación y un POST que cambia el estado

La eliminación es especialmente útil para enseñar que **los métodos HTTP tienen consecuencias**. Queremos permitir que el usuario abra una pantalla de confirmación sin borrar nada. Por eso, GET `/Productos/Eliminar/{id}` debe mostrar el nombre y el identificador, y ofrecer un botón explícito para confirmar la operación.

El GET de `Eliminar` únicamente lee la entidad:

```
[HttpGet]
public async Task<IActionResult> Eliminar(int id)
{
    if (id <= 0) return BadRequest();

    var producto = await _db.Productos.AsNoTracking()
        .FirstOrDefaultAsync(p => p.Id == id);

    return producto is null ? NotFound() : View(producto);
}
```

La vista `Eliminar.cshtml` muestra el producto y genera un formulario POST con Form Tag Helper, `asp-route-id` y el botón «Sí, eliminar». La navegación GET, la lectura de la vista y el botón «No, volver al catálogo» no son operaciones destructivas.

El POST usa la acción visible `Eliminar`, pero el método C# se llama `EliminarConfirmado` y contiene `[ActionName("Eliminar")]`. Este atributo permite exponer la misma operación de MVC con un nombre lógico coherente con el formulario:

```
[HttpPost, ActionName("Eliminar")]
[ValidateAntiForgeryToken]
public async Task<IActionResult> EliminarConfirmado(int id)
{
    if (id <= 0) return BadRequest();

    var producto = await _db.Productos
        .FirstOrDefaultAsync(p => p.Id == id);
    if (producto is null) return NotFound();
```

```
    _db.Productos.Remove(producto);
    await _db.SaveChangesAsync();
    return RedirectToAction(nameof(Index));
}
```

La secuencia separa la autorización del envío, la existencia del registro, el cambio de estado de EF Core y la operación persistente real. `Remove` marca la entidad para eliminarla; `SaveChangesAsync` aplica la operación en la base. Tras guardar, el navegador es redirigido al listado. Esto permite observar que el producto **ya no está** tanto en la lista como en su URL anterior de detalle, que ahora debería devolver 404.

Nunca enseñaremos una eliminación de datos reales que se ejecute al abrir un enlace GET. Es un ejemplo incorrecto de diseño HTTP y podría causar problemas con previsualizadores o herramientas que navegan automáticamente enlaces. El botón POST protegido y la página de confirmación tienen aquí un propósito educativo, además de su utilidad práctica.

Una eliminación que debe seguir eliminada tras reiniciar

La comprobación final de persistencia debe ir más allá de la respuesta inmediata del POST. Tras eliminar un producto de laboratorio, detén Kestrel, vuelve a iniciarlo, abre `/Productos` y verifica que **no reaparece**. Recuerda que la semilla de productos se ejecutó solamente cuando la base se creó por primera vez; el reinicio no vuelve a insertar todas las filas.

Si el producto reaparece, el diagnóstico debe comenzar por la conexión de base de datos utilizada en cada ejecución y por el código de inicialización, no por la vista Razor. También hay que evitar confundir dos bases con nombres distintos: cambiar `ConnectionStrings:Productos` entre arranques puede hacer que parezca que se han perdido o restaurado datos cuando en realidad estamos consultando almacenes diferentes.

La eliminación de un registro conocido puede probarse con una URL de detalle antes y después del POST: 200 antes, 404 después. Un nombre que deja de mostrarse en un fragmento visual aislado no constituye por sí solo una prueba tan sólida como la consulta al mismo identificador persistente.

Diferenciar las cuatro operaciones mediante HTTP y EF Core

Podemos resumir el itinerario de P2.4 como cuatro operaciones sobre una misma entidad, con responsabilidades diferentes:

Operación	Solicitud	Acción MVC	Trabajo EF Core	Resultado
Listar/ Detalle	GET	Index/ Detalle	Consulta, normalmente sin seguimiento	HTML de listado o ficha
Crear	GET, después POST	Crear	Add + SaveChangesAsync	302 a detalle del nuevo Id
Editar	GET, después POST	Editar	Consultar entidad y asignar campos + SaveChangesAsync	302 a detalle actualizado
Eliminar	GET, después POST	Eliminar	Buscar entidad + Remove + SaveChangesAsync	302 a listado

La tabla no debe memorizarse aisladamente. Su valor surge al ejecutarla paso a paso, observando las rutas y viendo cuándo cambia el contenido de LocalDB. **GET no realiza una escritura** en estos escenarios; el trabajo persistente se concentra en POST y está protegido con antiforgery.

La consulta `Index` utiliza `AsNoTracking()`, mientras que las modificaciones trabajan sobre entidades seguidas. Ese contraste ayuda a comprender por qué una página que muestra datos no tiene que alterar el estado del contexto, pero una edición necesita que los cambios queden registrados antes de guardarse.

Las tres operaciones que cambian datos aplican también una regla de navegación: tras guardar correctamente, el servidor responde con una redirección a una representación GET. La diferencia de destinos es razonable: después de crear o editar mostramos el detalle del producto identificado; después de eliminar volvemos al catálogo porque la ficha anterior ya no existe.

Antiforgery, ModelState y sobrepublicación siguen presentes

Una tentación habitual al conectar un formulario con EF Core es prestar atención solamente a la llamada `SaveChangesAsync` y olvidar las protecciones aprendidas. Eso sería retroceder pedagógicamente. La nueva operación persistente debe respetar **el mismo contrato de entrada** que ya se comprobó en P2.3 y los mecanismos antiforgery utilizados desde P2.1.

Los formularios de creación, edición y eliminación se generan con Form Tag Helper y un POST. El controlador mantiene `[ValidateAntiForgeryToken]`. La creación y la edición reciben `ProductoFormulario`, cuyas propiedades están restringidas por anotaciones, y consultan `ModelState.IsValid` antes de llamar a EF Core. En eliminación no existe un

formulario de nombre y precio; lo importante es que la solicitud incluya el Id de la ruta y un token válido, y que el servidor compruebe la existencia de la entidad.

Los límites del ejemplo también deben quedar claros. No estamos implementando una política de permisos: cualquier usuario que pueda navegar al formulario de laboratorio y obtener su token podría, en ausencia de autenticación/autorización adicional, utilizar sus operaciones. En una aplicación real habría que definir quién puede modificar o borrar productos. Ese tema está fuera del CRUD didáctico y será tratado superficialmente en el módulo de seguridad, sin pretender que este código esté listo para Internet.

Tampoco implementamos control de concurrencia optimista. Si dos operadores editan el mismo producto al mismo tiempo, podríamos necesitar marcas de versión y una política de conflictos. No vamos a entregar una falsa promesa de seguridad empresarial a cambio de unas líneas adicionales de ejemplo; estas limitaciones forman parte de la explicación honesta del alcance.

Amplificación razonada: un catálogo vacío y una búsqueda GET

Después de completar Create, Edit y Delete, el alumno puede desarrollar dos variantes opcionales que ponen a prueba su comprensión. No forman parte del recorrido imprescindible de las seis horas y no requieren añadir una nueva aplicación.

La **primera ampliación** se resuelve en Razor. Si se eliminan todos los registros del laboratorio, el listado `Index` seguirá recibiendo una colección válida, pero estará vacía. El código de presentación debe diferenciar `Model is null` de `Model.Count == 0`. Un modelo nulo significa que no se ha proporcionado la colección esperada; una colección vacía es un resultado perfectamente legítimo de una consulta. La actividad pide mostrar «No hay productos guardados» solo en el segundo caso.

La **segunda ampliación** es una búsqueda de productos por nombre. La solicitud no altera datos, por lo que tiene sentido utilizar GET:

```
GET /catalogo/buscar?nombre=Lapiz
```

El controlador contiene un bloque de reto comentado:

```
[HttpGet("/catalogo/buscar")]
public async Task<IActionResult> Buscar(string? nombre)
{
```

```
    var consulta = _db.Productos.AsNoTracking();

    if (!string.IsNullOrWhiteSpace(nombre))
        consulta = consulta.Where(p => p.Nombre.Contains(nombre));

    var encontrados = await consulta.OrderBy(p => p.Id).ToListAsync();
    return View("Index", encontrados);
}
```

La consulta se compone a partir de `IQueryable`; EF Core puede traducir la expresión al proveedor SQL Server. **No se cargan todos los productos para filtrarlos en memoria a mano**. Debemos recordar que la coincidencia textual puede depender de la intercalación o *collation* de la base: por ejemplo, las reglas de mayúsculas y acentos dependen de la configuración de SQL Server, no de un comportamiento universal de `Contains`.

La vista `Index` ya sabe representar una colección de productos, así que el método devuelve `View("Index", encontrados)`. No necesitamos construir otra lista visual. El proyecto incluye también un pequeño formulario GET comentado para remitir el término de búsqueda. Una vez activadas ambas partes, el alumno podrá comparar el catálogo completo con el resultado filtrado y confirmar que la consulta no ha modificado la persistencia.

La búsqueda es un ejemplo de **reutilización de lo aprendido**: routing, model binding de un parámetro simple, LINQ to Entities, lectura de EF Core y Razor. No introduce técnicas de búsqueda de texto completo ni optimización de índices, temas ajenos al alcance de esta jornada.

Diagnosticar lo que parece un error de MVC y en realidad es SQL

Con persistencia, una misma pantalla puede fallar por razones muy diferentes. Si la aplicación no arranca porque `EnsureCreated` no consigue conectarse, el problema aparece antes de que una solicitud HTTP llegue a los controladores. Debemos verificar LocalDB, la cadena de conexión y la instancia. Modificar `@model` de una vista no resolverá una instancia SQL inexistente.

Si el catálogo devuelve una excepción al consultar, preguntaremos si la base está disponible, si el esquema coincide con la entidad y si se creó correctamente. Si un formulario de creación muestra 405 en su estado inicial, es posible que el POST de P2.4.3 todavía esté comentado: esa observación pertenece a la secuencia de

aprendizaje. Si muestra 400 al enviar sin token, debemos revisar antiforgery; no implica que SQL Server haya rechazado los datos.

Si un formulario inválido devuelve 200 y se vuelven a mostrar sus mensajes, la aplicación puede estar funcionando exactamente como se diseñó: `ModelState` impidió la escritura. Lo que no debería ocurrir es que el dato inválido aparezca en el catálogo. En ese caso habría que comprobar la posición de `SaveChangesAsync` respecto de la condición de validación.

Si la edición vuelve a mostrarse con un error pero pierde el Id del producto, el síntoma apunta a la generación de la ruta del formulario después de `return View(entrada)`. El controlador necesita conservar el identificador que permite volver a editar la misma entidad. Si después de eliminar una fila su detalle sigue devolviendo 200, necesitamos investigar si se ejecutó el POST adecuado, qué base de datos se está consultando y si se guardó realmente la operación.

El mejor diagnóstico sigue el orden **solicitud → acción → validación → contexto EF → persistencia → respuesta → nueva consulta**. No consiste en recompilar de manera repetida sin identificar qué capa produce el comportamiento inesperado.

Cierre de 2.4 y preparación de Razor Pages

El resultado de P2.4 es la misma aplicación MVC de M01, ahora capaz de representar datos que viven en una base LocalDB y de modificar esa base mediante formularios validados. Hemos empleado C# y EF Core que el alumnado ya conoce, pero los hemos integrado en un recorrido de interfaz web que ahora incluye redirecciones, mensajes de validación, enlaces con Id, confirmación de borrado y comprobaciones posteriores al reinicio.

Al iniciar **2.5** estudiaremos una Razor Page mínima dentro del mismo host. No volveremos a enseñar un segundo CRUD completo ni cambiaremos la fuente de datos: la finalidad será comparar **controlador + vista MVC** con **PageModel + OnGet/OnPost**. El ejemplo será breve precisamente porque este CRUD concentra la mayor parte del trabajo aplicado del día.

El criterio de salida para 2.4 debe poder expresarse con ejemplos concretos: crear un producto válido, observar su nueva ficha, reiniciar y comprobar que permanece; editar el mismo Id y constatar el cambio; intentar una entrada inválida y comprobar que no modifica la base; eliminar mediante POST y verificar que el detalle ya no

existe. Esas observaciones demuestran mucho más que una pantalla que dice «operación correcta».

Referencias de ampliación

- Microsoft Learn, CRUD en una aplicación ASP.NET Core MVC con EF Core: https://learn.microsoft.com/en-us/aspnet/core/data/ef-mvc/crud?view=aspnetcore-8.0
- Microsoft Learn, Creación y administración del esquema sin migraciones: https://learn.microsoft.com/en-us/ef/core/managing-schemas/ensure-created
- Microsoft Learn, Registro de DbContext e inyección de dependencias: https://learn.microsoft.com/en-us/ef/core/dbcontext-configuration/
- Microsoft Learn, Consultas sin seguimiento en EF Core: https://learn.microsoft.com/en-us/ef/core/querying/tracking
- Microsoft Learn, Protección antiforgery de solicitudes MVC: https://learn.microsoft.com/en-us/aspnet/core/security/anti-request-forgery?view=aspnetcore-8.0

Profundización aplicada: distinguir pantalla, entidad y transacción

En el CRUD persistente conviven varias representaciones de un producto que conviene no confundir. El formulario HTML contiene valores escritos como texto. `ProductoFormulario` representa esos valores una vez enlazados y validados. `Producto` es la entidad que EF Core traduce a un registro persistente. Finalmente, la vista de detalle representa una instantánea de lectura devuelta al usuario. Aunque las cuatro representaciones puedan mostrar «Rotulador», no son el mismo objeto ni producen los mismos efectos. Esta distinción será importante cuando modifiquemos solo el precio de un producto y queramos garantizar que no se crea un registro nuevo.

En una lectura destinada exclusivamente a representar información, `AsNoTracking` comunica que no necesitamos conservar seguimiento de entidades para cambios posteriores dentro de ese contexto. En cambio, la acción de edición recupera la entidad que va a modificar, asigna nuevos valores y llama a `SaveChangesAsync`. Ese recorrido aprovecha el seguimiento de EF Core. No necesitamos impartir de nuevo toda la maquinaria del cambio de estados; sí es esencial saber por qué editar una entidad existente no equivale a crear una nueva con un identificador arbitrario recibido del navegador.

Una acción POST puede haber modificado la base y, sin embargo, el navegador continuar mostrando una respuesta anterior si no ejecutamos una lectura posterior. De ahí la utilidad del patrón **Post/Redirect/Get** empleado tras crear, editar y borrar: el servidor termina la operación, responde con una redirección y el navegador solicita por GET la representación resultante. La redirección no garantiza por sí sola que se hayan guardado datos: esa garantía depende de que `SaveChangesAsync` haya completado correctamente el trabajo. Lo importante es que separamos la solicitud de escritura de la nueva consulta que permite observar el estado persistido.

Los identificadores también pertenecen a dominios distintos. El valor `id` que aparece en una ruta permite localizar una entidad; no es automáticamente un permiso para editarla. En un producto de laboratorio, basta comprobar que el Id tiene sentido y que existe una fila. En una aplicación real necesitaríamos además comprobar qué usuario está autorizado. No añadimos aquí un sistema completo de seguridad, pero conviene evitar una conclusión incorrecta: antiforgery protege frente a una clase concreta de solicitudes falsificadas, mientras que la autorización decide quién puede realizar una operación sobre un recurso.

LocalDB añade condiciones de entorno que no son defectos del patrón MVC. La cadena de conexión identifica instancia y base; `EnsureCreated` intenta disponer del esquema en un laboratorio nuevo; la semilla inicial se inserta solo cuando corresponde al primer arranque de esa base. Si alguien utiliza otra cadena de conexión después del ejercicio, podría ver un catálogo aparentemente «reiniciado» aunque la primera base conserve sus registros. Por eso las prácticas recomiendan trabajar con una base propia y comprobar los datos antes y después de reiniciar **el mismo host con la misma conexión**.

En la edición de un producto con Id 7 pueden darse varios resultados. Si el GET no encuentra la entidad, devolver 404 es coherente porque la dirección señala un recurso inexistente. Si el GET la encuentra pero el POST trae un nombre inválido, volver a presentar la vista es coherente porque el usuario necesita corregirlo. Si el POST es válido, recuperar la entidad por Id y aplicar los cambios permite guardar en esa fila concreta. Si intentamos editar con un Id incorrecto, no deberíamos asignar silenciosamente los valores a otra fila. Cada resultado tiene una causa distinta y ofrece una pista diagnóstica.

La eliminación muestra claramente por qué GET y POST tienen papeles diferentes. El GET de confirmación **no borra** nada: presenta el nombre del producto y pregunta

al usuario si desea continuar. El POST de confirmación, protegido con antiforgery, puede eliminar la entidad y redirigir al catálogo. Si tras un GET la ficha deja de estar disponible, hay un error grave de diseño; si tras un POST correcto continúa apareciendo, debemos verificar el Id, la base y la ejecución de `SaveChangesAsync`. Esta observación es una forma sencilla de comprobar que la interfaz no es solo una representación que oculta filas visualmente.

Perspectiva profesional sin ampliar el ejercicio. Los sistemas reales suelen necesitar control de concurrencia, manejo de excepciones transitorias, autorización y migraciones de esquema. No implementamos esas capacidades en esta jornada, pero sabemos identificar sus límites: `EnsureCreated` es una comodidad de laboratorio y no un sustituto general de las migraciones; la búsqueda opcional no constituye un motor de texto completo; los precios no incluyen todavía reglas fiscales ni monetarias complejas. Reconocer un límite explícito evita enseñar como patrón universal lo que se ha simplificado deliberadamente para aprender el recorrido MVC–EF Core.

2.5. Razor Pages: otra organización para el mismo ciclo HTTP

Del catálogo MVC a una página centrada en sí misma

En 2.4 terminamos un recorrido completo: el navegador envía una petición, el enrutamiento selecciona una acción de `ProductosController`, la acción consulta o modifica el catálogo mediante EF Core y una vista Razor produce HTML. Para crear un producto, incluso en el caso más pequeño, intervienen al menos una acción GET, una acción POST y la vista `Views/Productos/Crear.cshtml`. Esa separación resulta útil cuando coordinamos varios recursos y operaciones, pero no es la única forma de organizar una interfaz web que recibe y representa datos.

Razor Pages proporciona una alternativa integrada en ASP.NET Core para construir interfaces organizadas por página. En lugar de asociar la solicitud a una acción de un controlador MVC y después seleccionar una vista, asociamos una dirección a un archivo de página `.cshtml` y a sus métodos controladores de petición, denominados *handlers*. Podemos utilizar ambos estilos dentro de la misma aplicación, con las mismas ideas de HTTP, Razor, model binding, validación, Tag Helpers y protección antiforgery.

No estamos cambiando la tecnología de servidor ni construyendo Blazor. Seguimos recibiendo HTTP y generando HTML. Razor Pages utiliza los servicios e infraestructura de ASP.NET Core MVC, pero propone una **organización orientada a páginas**. Tampoco obliga a prescindir de servicios, EF Core o separación de responsabilidades en aplicaciones mayores; en este ejercicio limitamos deliberadamente la página a un nombre y un mensaje.

Una solicitud, dos formas de distribuir la responsabilidad

En MVC, la solicitud `GET /Productos/Crear` se resuelve mediante una acción semejante a:

```
[HttpGet]
public IActionResult Crear() => View(new ProductoFormulario());
```

Después, el controlador devuelve una vista en `Views/Productos/Crear.cshtml`. La vista lleva `@model` y dibuja el formulario, pero no decide por sí sola qué acción HTTP atenderá la ruta. En el POST real de 2.4, otra sobrecarga de `Crear` recibe

`ProductoFormulario`, comprueba `ModelState` y, cuando procede, escribe en LocalDB mediante el contexto EF.

En P2.5 mantendremos ese catálogo sin modificarlo. Incorporamos únicamente dos archivos nuevos que funcionan como pareja:

```
CursoWeb.Mvc/
  Pages/
    Comparacion.cshtml
    Comparacion.cshtml.cs
    _ViewImports.cshtml
    _ViewStart.cshtml
  Controllers/
    ProductosController.cs
  Views/
    Shared/
      _Layout.cshtml
```

El primero contiene el marcado Razor y comienza así:

```
@page "/Comparacion"
@model CursoWeb.Mvc.Pages.ComparacionModel
@{
    ViewData["Title"] = "MVC y Razor Pages";
}
<h1>P2.5 | Comparación entre MVC y Razor Pages</h1>
<p id="mensaje-page">@Model.Mensaje</p>
```

La directiva **`@page`** convierte el archivo Razor en un endpoint de Razor Pages; el texto `"/Comparacion"` fija su ruta, de modo que podemos solicitar `GET /Comparacion`. **`@model`** indica el tipo de la clase que respalda la página. No debemos confundir esta página con una vista MVC normal: aunque ambas usan archivos `.cshtml` y expresiones Razor, una vista MVC es seleccionada por una acción y no se vuelve direccionable por añadirle un nombre de archivo; una Razor Page se identifica como endpoint gracias a `@page` y al mapeo correspondiente.

Esta comparación prepara además M03: allí aparecerán **componentes Razor de Blazor**. Compartir la extensión `.razor` o la sintaxis de expresiones Razor no implica compartir el mismo mecanismo de ejecución. Una Razor Page sirve una respuesta de página ante GET o POST; la interactividad de Blazor necesita además un modo de renderizado interactivo y un ciclo de eventos de componentes.

El mismo host registra MVC y Razor Pages

La coexistencia no requiere una segunda aplicación web. Nuestro archivo real `Program.cs`, en `PROYECTO/2.5/CursoWeb.Mvc`, registra ambos estilos:

```
builder.Services.AddControllersWithViews();
builder.Services.AddRazorPages();
```

Al configurar los endpoints, el mismo host contiene:

```
app.UseStaticFiles();
app.UseRouting();
app.MapRazorPages();
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
```

`AddRazorPages` registra los servicios necesarios y `MapRazorPages` publica los endpoints de las páginas; `AddControllersWithViews` y `MapControllerRoute` conservan el funcionamiento del catálogo. Si existe `Comparacion.cshtml` pero omitimos el mapeo de Razor Pages, la ruta no quedará disponible como pretendemos. Si omitimos el mapeo convencional de MVC, el catálogo existente perderá sus rutas habituales.

La aplicación sigue configurando `AppDbContext` con SQL Server Express LocalDB y ejecutando `EnsureCreated()` al iniciar, tal como hacía P2.4. **Esto significa que el host todavía necesita una instancia LocalDB accesible para arrancar**, aunque la página `/Comparacion` no realice ninguna consulta ni escritura de productos. No hay que atribuir a Razor Pages un error de arranque provocado por la conexión a SQL Server.

PageModel: dónde viven los handlers y el estado del formulario

El archivo `Pages/Comparacion.cshtml.cs` contiene:

```
public sealed class ComparacionModel : PageModel
{
    [BindProperty]
    public string? Nombre { get; set; }

    public string Mensaje { get; private set; } = string.Empty;

    public void OnGet()
    {
        Mensaje = "OnGet prepara la página sin enviar un formulario.";
```

```
    }
}
```

La clase hereda de **`PageModel`**, el objeto que organiza el comportamiento de la página. El `.cshtml.cs` es su archivo *code-behind* y comparte el nombre base de la página. En el ejemplo, `Nombre` recibe la entrada del formulario; `Mensaje` es una propiedad que la página lee para mostrar el resultado. `Mensaje` no es un campo de base de datos ni un valor persistente: pertenece a la instancia de la solicitud actual.

El handler **`OnGet`** se ejecuta al atender un GET que coincide con esta página. Escribe un mensaje en `Mensaje`; después Razor evalúa `@Model.Mensaje` y genera el HTML que recibe el navegador. Si el usuario vuelve a solicitar la dirección, se crea un nuevo procesamiento de petición. No hemos implementado ninguna sesión ni un almacenamiento compartido de mensajes.

La coincidencia entre verbo HTTP y nombre de handler es intencionada: `OnGet` para GET, `OnPost` para POST. No es necesario convertir cada handler en una acción de un controlador diferente. Razor Pages puede tener handlers síncronos o asíncronos; en esta demostración, no realizamos I/O que justifique emplear `OnPostAsync`.

Del GET al POST: enviar un nombre y volver a representar la página

El marcado actual contiene este formulario:

```
<form method="post" asp-antiforgery="true">
    <div asp-validation-summary="ModelOnly" class="text-danger"></div>
    <div>
        <label asp-for="Nombre"></label>
        <input asp-for="Nombre" />
        <span asp-validation-for="Nombre" class="text-danger"></span>
    </div>
    <button type="submit">Enviar a OnPost</button>
</form>
```

El formulario envía un POST hacia la misma página, no hacia `ProductosController.Crear`. El atributo `asp-for` conecta la etiqueta, el campo y el mensaje de validación con `Nombre`. Igual que en los ejemplos MVC, los **Tag Helpers se ejecutan en el servidor** y generan HTML ordinario. El archivo `Pages/_ViewImports.cshtml` contiene `@addTagHelper *, Microsoft.AspNetCore.Mvc.TagHelpers`, necesario para su aplicación.

La propiedad está marcada con **`[BindProperty]`**, por lo que los datos enviados en el formulario pueden enlazarse con `Nombre` en el handler POST. Esto no significa que

cualquier GET enlace automáticamente esa propiedad desde la URL. Para enlazar propiedades de página en GET se necesitaría una decisión explícita como `SupportsGet = true`; aquí no la necesitamos.

El proyecto inicial entrega el método `OnPost` dentro del bloque `PASO 2.5.3`, desactivado para que el alumno lo habilite después de observar `OnGet`. Su versión preparada es:

```
public IActionResult OnPost()
{
    if (!ModelState.IsValid)
        return Page();

    if (string.IsNullOrWhiteSpace(Nombre))
    {
        ModelState.AddModelError(nameof(Nombre), "Introduce un nombre.");
        return Page();
    }

    Mensaje = $"OnPost ha recibido: {Nombre.Trim()}";
    return Page();
}
```

Al activarlo, el recorrido es: el navegador envía los campos, Razor Pages selecciona el handler POST de `ComparacionModel`, el binder asigna `Nombre`, la validación actualiza `ModelState`, el handler decide qué respuesta producir y `Page()` vuelve a representar `Comparacion.cshtml`. No hemos guardado nada: no existe ningún `_db.Productos.Add` ni `SaveChangesAsync` en ese handler.

La llamada **`return Page()`** es conceptualmente distinta de `return View(entrada)` en MVC, aunque ambas terminan generando HTML. La primera devuelve la Razor Page actual con su `PageModel`; la segunda devuelve una vista MVC elegida por una acción. También sería posible redirigir tras un POST, pero aquí **no lo hacemos** porque queremos observar el mensaje de la misma petición.

Una precaución de diagnóstico: no debemos enseñar que un POST sin `OnPost` devuelve necesariamente 405. En Razor Pages la ausencia de un handler explícito puede producir el renderizado de la propia página sin haber ejecutado nuestra lógica de POST. Lo relevante del estado inicial es comprobar que **no aparece el mensaje `OnPost ha recibido`**. Una vez activado el handler, la nueva conducta sí puede comprobarse con un nombre concreto.

DataAnnotations y ModelState siguen significando lo mismo

El formulario de P2.5 utiliza los mecanismos que ya estudiamos en 2.3, pero aplicados a una propiedad del `PageModel`. El código contiene estas anotaciones preparadas para activar en el paso 2.5.4:

```
[BindProperty]
// [Required(ErrorMessage = "Introduce un nombre.")]
// [StringLength(60, MinimumLength = 2,
//     ErrorMessage = "El nombre debe tener entre 2 y 60 caracteres.")]
public string? Nombre { get; set; }
```

Al retirar únicamente los prefijos `//` de las dos anotaciones, una entrada ausente debe generar el error de `Required` y un nombre de un solo carácter debe generar el error de `StringLength`. Un nombre correcto permite alcanzar la asignación a `Mensaje`. El modelo admite `string?` para no ocultar que el valor puede estar ausente antes de pasar la validación; la regla `Required` es la que expresa el requisito de negocio.

Como en MVC, **`ModelState` no es el nombre de un validador específico**, sino el conjunto de valores y errores resultante del enlace y la validación de la solicitud. Cuando el handler devuelve `Page()` ante una entrada inválida, los Tag Helpers representan los mensajes y conservan el dato del campo, para que el usuario pueda corregirlo.

Hay una sutileza útil: `asp-validation-summary="ModelOnly"` muestra errores de nivel de modelo, mientras que `asp-validation-for="Nombre"` es el lugar previsto para el error de esta propiedad. Si esperamos leer allí un fallo de `Required` y solo inspeccionamos el resumen, interpretaremos incorrectamente una página que funciona.

P2.5 comprueba principalmente **validación de servidor**. El marcado de `asp-validation-for` no demuestra por sí solo que los scripts de validación de navegador estén cargados. La validación JavaScript se trató en 2.3; no es requisito volver a instalar ni desarrollar esos scripts durante los quince minutos de esta práctica.

Antiforgery: un POST de Razor Pages tampoco es una excepción de seguridad

Un navegador conserva cookies de una sesión entre solicitudes. Sin protección, una página ajena podría intentar provocar un POST hacia nuestra aplicación usando ese contexto. Por eso estudiamos antiforgery en los formularios MVC. Razor Pages mantiene la misma preocupación: los formularios POST de páginas están protegidos frente a falsificación de solicitudes de forma predeterminada.

El ejemplo escribe `asp-antiforgery="true"` para hacer explícita la generación del campo oculto de protección por el Form Tag Helper. Al inspeccionar el HTML se debe encontrar `__RequestVerificationToken`. Para que la verificación tenga sentido también interviene el contexto de cookies de la petición original. Un POST fabricado sin token debe rechazarse antes de ejecutar el handler protegido; no basta con enviar `Nombre` correctamente.

No hay que colocar dos mecanismos de generación del campo por costumbre. En este proyecto **el Form Tag Helper genera el token**; no se añade simultáneamente `@Html.AntiForgeryToken()`. Tampoco se desactiva la protección para simplificar una prueba. La comparación importante con 2.4 es que MVC marca sus acciones POST con `[ValidateAntiForgeryToken]`, mientras que Razor Pages aplica por defecto la protección a sus handlers de métodos no seguros.

Un HTTP 400 sin token y un HTTP 200 que vuelve a mostrar un error de `ModelState` representan causas distintas. El primer resultado procede del filtro de seguridad; el segundo puede ser la respuesta correcta de un formulario validado. Confundirlos lleva a modificar el componente equivocado.

Más de un POST en una página: handlers con nombre

Una Razor Page no está limitada a `OnGet` y `OnPost`. El reto opcional prepara otro handler:

```
public IActionResult OnPostMayusculas()
{
    if (!ModelState.IsValid)
        return Page();

    if (string.IsNullOrWhiteSpace(Nombre))
    {
        ModelState.AddModelError(nameof(Nombre), "Introduce un nombre.");
        return Page();
    }

    Mensaje = $"Nombre en mayúsculas: {Nombre.Trim().ToUpperInvariant()}";
    return Page();
}
```

El nuevo nombre responde a la convención `On` + verbo `Post` + handler `Mayusculas`. Para solicitarlo, la vista incorpora este botón alternativo:

```
<button type="submit" asp-page-handler="Mayusculas">Mostrar en mayúsculas</button>
```

Al generar HTML, el Tag Helper construye el destino del handler, habitualmente a través de un valor de ruta o consulta como `?handler=Mayusculas`, según la plantilla de ruta. No se debe escribir `OnPostMayusculas` en el atributo ni convertirlo en un endpoint MVC. El botón predeterminado sigue ejecutando `OnPost`.

Los dos handlers comparten las reglas de validación de `Nombre` y la protección antiforgery. Usamos `ToUpperInvariant()` para mostrar una transformación estable en este pequeño ejemplo; eso no equivale a implementar reglas lingüísticas completas para todos los idiomas. Tampoco se actualiza ningún producto de la base de datos.

Navegación cruzada y layout compartido

En `Pages/_ViewStart.cshtml` se configura:

```
@{
    Layout = "/Views/Shared/_Layout.cshtml";
}
```

Así, la Razor Page usa el layout ya conocido por el alumnado: cabecera, navegación y estructura visual se mantienen. El layout contiene un enlace preparado para el paso 2.5.5:

```
<a asp-page="/Comparacion">Comparar Razor Pages</a>
```

`asp-page` construye un enlace hacia la Razor Page; para regresar al catálogo desde `Comparacion.cshtml`, utilizamos el mecanismo MVC que ya conocemos:

```
<a asp-controller="Productos" asp-action="Index">Volver al catálogo MVC</a>
```

Los enlaces no hacen que una Razor Page se convierta en vista MVC ni viceversa. Son simplemente direcciones generadas por Tag Helpers para endpoints distintos de un mismo host. La dirección `/Comparacion` sigue correspondiendo a `@page`; `/Productos` sigue llegando a `ProductosController`.

Cuándo resulta razonable elegir cada estilo

Una aplicación con pantallas relativamente independientes, cada una con un formulario y su propia lógica de carga y envío, puede encajar bien en Razor Pages: el código de esa pantalla queda próximo a su marcado. Un sistema con varias acciones sobre un recurso, controladores compartidos, rutas organizadas alrededor de operaciones y vistas reutilizables puede resultar natural en MVC. Ninguna de

esas descripciones constituye una prohibición: ambas alternativas pueden utilizar servicios, persistencia, vistas parciales y pruebas.

En nuestro curso la elección está guiada por el objetivo formativo. **M01 y M02 usan MVC** para aprender el patrón y construir el CRUD. **P2.5 usa una sola Razor Page** para reconocer el contraste. **M03 comenzará Blazor**, cuyo modelo de eventos y componentes interactivos es otro concepto; no añadimos aquí esa complejidad.

Diagnóstico de una página que no se comporta como esperamos

Si `/Comparacion` responde 404, primero revisaremos `@page "/Comparacion"`, `AddRazorPages`, `MapRazorPages` y la carpeta `Pages`. No sirve renombrar acciones de `ProductosController` para arreglar el endpoint de página.

Si la aplicación no arranca y el error menciona SQL Server o `EnsureCreated`, la causa probable es el entorno LocalDB heredado de 2.4; no deduciremos que Razor Pages ha fallado al resolver `OnGet`.

Si el GET muestra el mensaje pero un POST no produce `OnPost ha recibido`, comprobar primero si el bloque del handler permanece comentado, después el método del formulario y, por último, los errores de modelo. Un POST sin token puede devolver 400 **antes** de llegar al handler.

Si un nombre corto sigue siendo aceptado, comprobar que se han activado las dos anotaciones de 2.5.4 y que se recompiló la aplicación. Si el error existe pero no se ve en el resumen superior, mirar `asp-validation-for="Nombre"`, que representa los errores de propiedad. Si el valor se pierde en una página inválida, verificar que devolvemos `Page()` y conservamos `ModelState` sin redirigir ni sobrescribir `Nombre`.

Si el botón de mayúsculas no llama al handler previsto, verificar conjuntamente el método `OnPostMayusculas`, el `asp-page-handler="Mayusculas"` y el HTML generado; la existencia de uno solo de esos elementos no completa el reto. Si la navegación aparece como texto literal `asp-page`, comprobar `_ViewImports.cshtml`.

Por último, la comprobación de no regresión debe ser sencilla: la Razor Page no escribe en LocalDB. Tras visitar la página y enviar nombres, el catálogo MVC debe seguir conteniendo los mismos productos. Un cambio en el catálogo durante este ejercicio indicaría una modificación imprevista en otro punto del proyecto.

Cierre del módulo y lecturas recomendadas

Hemos seguido un mismo hilo desde 2.1: primero aprendimos a **enviar** valores, después a generar formularios y enlaces mantenibles, más tarde a **validar** entradas y, finalmente, a modificar un catálogo persistente. Razor Pages vuelve al inicio de ese hilo para enseñarnos que podemos organizar de otra forma el código que recibe el formulario, sin abandonar HTTP ni las protecciones que ya conocemos. El alumno debe terminar pudiendo explicar, delante de las dos pantallas, quién atiende cada GET y cada POST, qué representa `ModelState` y por qué ninguna operación de esta página modifica el catálogo.

El recorrido principal de P2.5 es breve. Los detalles sobre handlers adicionales, validación negativa y criterios de elección sirven para estudio autónomo y no añaden otro bloque obligatorio de programación a la jornada.

- Microsoft Learn, Introducción a Razor Pages: https://learn.microsoft.com/es-es/aspnet/core/razor-pages/?view=aspnetcore-8.0
- Microsoft Learn, Introducción a Razor Pages con ASP.NET Core: https://learn.microsoft.com/es-es/aspnet/core/tutorials/razor-pages/razor-pages-start?view=aspnetcore-8.0
- Microsoft Learn, Convenciones de autorización y rutas de Razor Pages: https://learn.microsoft.com/en-us/aspnet/core/razor-pages/razor-pages-conventions?view=aspnetcore-8.0
- Microsoft Learn, Form Tag Helper y antiforgery: https://learn.microsoft.com/en-us/aspnet/core/mvc/views/working-with-forms?view=aspnetcore-8.0
- Microsoft Learn, Model binding en ASP.NET Core: https://learn.microsoft.com/en-us/aspnet/core/mvc/models/model-binding?view=aspnetcore-8.0